

Государственное бюджетное профессиональное образовательное учреждение
Краснодарского края «Армавирский юридический техникум»

(ГБПОУ КК АЮТ)

Гаврющенко А.А.

ОП.01 ОПЕРАЦИОННЫЕ СИСТЕМЫ И СРЕДЫ

курс лекций

**для обучающихся 2 курса дневной формы обучения
специальности 09.02.06 Сетевое и системное
администрирование**

2022-2023 уч. год

УДК 004(075.32)

ББК 32.973-018.2я723

Г-12

Рассмотрено и рекомендовано
на заседании цикловой методической
комиссии

«__»_____2023 г.

Протокол №_____

Председатель ЦМК

_____ О.А. Волкодамова

Одобрено методическим
советом техникума

«__»_____2023 г.

Протокол №_____

Председатель методсовета

_____ И.Г. Рачкова

Гаврющенко А.А. Курс лекций по дисциплине «Операционные системы и среды» / А.А. Гаврющенко, Армавир, 2023. - Текст: непосредственный.

Данные материалы предназначены для обеспечения лекционных занятий по дисциплине Операционные системы и среды студентами 2 курса по специальности 09.02.06 Сетевое и системное администрирование.

Материалы охватывают все темы дисциплины Операционные системы и среды могут использоваться в качестве конспекта лекций.

Для студентов специальности 09.02.06 Сетевое и системное администрирование.

Рецензент:

Волкодамова О.А. – преподаватель ГБПОУ КК АЮТ

Ответственные за выпуск:

Тагирова А.Х. – методист ГБПОУ КК АЮТ

© ГБПОУ КК АЮТ, 2023

© Гаврющенко А.А., 2023

Содержание

Введение.....	4
Лекция 1	5
Лекция 2	9
Лекция 3	12
Лекция 4	22
Лекция 5	26
Лекция 6	30
Лекция 7	39
Лекция 8	46
Лекция 9	53
Лекция 10	57
Лекция 11	64
Лекция 12	69
Список использованных источников.....	74

Введение

Современный компьютер - универсальное, многофункциональное автоматическое устройство обработки информации. Сегодня компьютеры принимают на себя основную часть функций по обработке данных. По историческим меркам компьютерные технологии все еще молоды и находятся в самом начале развития, преобразуя или вытесняя традиционные информационные технологии.

Программное обеспечение (ПО) компьютера разделяется на общесистемное и прикладное.

Операционная система (ОС), являясь основой общесистемного ПО, обеспечивает функционирование и взаимосвязь всех компонентов компьютера и предоставляет пользователю свободный доступ к его аппаратным возможностям.

Прикладное программное обеспечение в свою очередь делится на средства разработки и приложения. Средства разработки — это инструменты программиста, включающие алгоритмические языки программирования (ЯП), а также трансляторы (компиляторы). Приложения - программные продукты, предназначенные для решения задач в конкретной предметной области.

В конспекте лекции «Операционные системы» изучаются основополагающие принципы устройства операционных систем, возможности применения фундаментальных концепций от достигнутого технологического уровня и специфических требований к конкретной реализации, их взаимосвязь с различными новациями в этой области, а также с современными направлениями развития операционных систем.

Тема 1.1. История, назначение и функции операционных систем

Лекция 1

История, назначение, функции и виды операционных систем

Первые (1945-1955г.г.) компьютеры работали без операционных систем, как правило, на них работала одна программа.

Когда скорость выполнения программ и их количество стало увеличиваться, простой компьютера между запусками программ стали составлять значительное время. Появились первые системы **пакетной обработки** (1955-1965г.г.), которые просто автоматизировали запуск одной программ за другой и тем самым увеличивали коэффициент загрузки процессора. Системы пакетной обработки явились прообразом современных операционных систем. Совокупность нескольких заданий, как правило, в виде колоды перфокарт, получила название пакета заданий.

Многозадачность (1965-1980) - это способ организации вычислительного процесса, при котором на одном процессоре попеременно выполняются несколько задач. Пока одна задача выполняет операцию ввода-вывода, процессор не простаивает, как это происходило при последовательном выполнении задач, а выполняет другую задачу. Для этого создали систему распределения памяти, когда каждая задача загружается в свой участок оперативной памяти, называемый разделом.

Спулинг (spooling-подкачка) в то время задания считывались с перфокарт на диск в том темпе, в котором они появлялись в помещении вычислительного центра, а затем, когда очередное задание завершалось, новое задание с диска загружалось в освободившийся раздел.

Системы разделения времени - вариант многозадачности, при котором у каждого пользователя есть свой диалоговый терминал. Это было сделано, чтобы каждый программист мог отлаживать свою программу в реальном времени. Фактически это была многопользовательская система. Естественно стали возникать проблемы защиты задач друг от друга.

В это время была разработана многопользовательская система MULTICS, которая должна была обеспечивать одновременную работу сотни пользователей.

В это время также стали бурно развиваться мини-компьютеры (первый был выпущен в 1961г.), на которые была перенесена система MULTICS. Эта работа в дальнейшем развилась в систему **UNIX**.

Появилось много разновидностей несовместимых UNIX, основные из них были System V и BSD. Чтобы было возможно писать программы, работающие в любой системе UNIX, был разработан стандарт **POSIX**. Стандарт POSIX определяет минимальный интерфейс системного вызова, который должны поддерживать системы UNIX.

В 1974г. был выпущен центральный процессор Intel 8080, для него была создана операционная система **CP/M**. В 1977г. она была переработана для других процессоров, например Z80.

В начале 80-х была разработана система **MS-DOS**, и стала основной системой для микрокомпьютеров.

В 80-х годах стало возможным реализовать **графический интерфейс пользователя** (GUI - GraphicalUserInterface), теория которого была разработана еще в 60-е годы. Первой реализовала GUI корпорация Macintosh.

С 1985 года выпустили **Windows**, в то время она была графической оболочкой к MS-DOS вплоть до 1995г., когда вышла **Windows 95**.

Уже тогда было ясно, что DOS с ее ограничениями по памяти и по возможностям файловой системы не может воспользоваться вычислительной мощностью появляющихся компьютеров. Поэтому IBM и Microsoft начинали совместно разрабатывать операционную систему **OS/2**. Она должна была поддерживать вытесняющую многозадачность, виртуальную память, графический пользовательский интерфейс, виртуальную машину для выполнения DOS-приложений. Первая версия вышла 1987г.

В дальнейшем Microsoft отошла от разработки OS/2, и стала разрабатывать **Windows NT**. Первая версия вышла в 1993г.

В середине 80-х стали бурно развиваться сети персональных компьютеров, работающие под управлением сетевых или распределенных операционных систем.

Сетевая операционная система не имеет отличий от операционной системы однопроцессорного компьютера. Она обязательно содержит программную поддержку для сетевых интерфейсных устройств (драйвер сетевого адаптера), а также средства для удаленного входа в другие компьютеры сети и средства доступа к удаленным файлам.

Распределенная операционная система, напротив, представляется пользователям простой системой, в которой пользователь не должен беспокоиться о том, где работают его программы или где расположены файлы, все это должно автоматически обрабатываться самой операционной системой.

В 1987г. была выпущена операционная система **MINIX**, она была построена на схеме **микро ядра**.

В 1991г. была выпущена **LINUX**, в отличии от микроядерной MINIX она стала монолитной.

Чуть позже вышла **FreeBSD** (основой для нее послужила BSD UNIX).

Назначение ОС

ОС как виртуальная машина

ОС предоставляет пользователю **виртуальную машину**, которую легче программировать и с которой легче работать, чем непосредственно с аппаратурой, составляющей реальную машину.

Например, чтобы считать или записать информацию на дискету, надо:

- Запустить двигатель вращения дискеты
- Управлять шаговым двигателем перемещения головки
- Следить за индикатором присутствия дискеты
- Выбрать номер блока на диске
- Выбрать дорожку
- Выбрать номер сектора на дорожке
- и т.д.

Все эти функции берет на себя операционная система.

ОС как система управления ресурсами

Чтобы несколько программ могло работать с одним ресурсом (процессор, память), необходима **система управления ресурсами**.

Способы распределения ресурса:

- **Временной** - когда программы используют его по очереди, например, так система управляет процессором.
- **Пространственный** - программа получает часть ресурса, например, так система управляет оперативной памятью и жестким диском.

Тема 1.2 Архитектура операционной системы

Лекция 2

Архитектура операционной системы

Под архитектурой операционной системы понимают структурную и функциональную организацию ОС на основе некоторой совокупности программных модулей. В состав ОС входят исполняемые и объектные модули стандартных для данной ОС форматов, программные модули специального формата (например, загрузчик ОС, драйверы ввода-вывода), конфигурационные файлы, файлы документации, модули справочной системы и т.д.

На архитектуру ранних операционных систем обращалось мало внимания: во-первых, ни у кого не было опыта в разработке больших программных систем, а во-вторых, проблема взаимозависимости и взаимодействия модулей недооценивалась. В подобных монолитных ОС почти все процедуры могли вызывать одна другую. Такое отсутствие структуры было несовместимо с расширением операционных систем. Первая версия ОС OS/360 была создана коллективом из 5000 человек за 5 лет и содержала более 1 млн строк кода. Разработанная несколько позже операционная система Mastics содержала к 1975 году уже 20 млн строк. Стало ясно, что разработка таких систем должна вестись на основе модульного программирования.

Большинство современных ОС представляют собой хорошо структурированные модульные системы, способные к развитию, расширению и переносу на новые платформы. Какой-либо единой унифицированной архитектуры ОС не существует, но известны универсальные подходы к структурированию ОС. Принципиально важными универсальными подходами к разработке архитектуры ОС являются:

- модульная организация;
- функциональная избыточность;
- функциональная избирательность;
- параметрическая универсальность;
- концепция многоуровневой иерархической вычислительной системы, по которой ОС представляется многослойной структурой;
- разделение модулей на две группы по функциям: ядро – модули, выполняющие основные функции ОС, и модули, выполняющие вспомогательные функции ОС;

- разделение модулей ОС на две группы по размещению в памяти вычислительной системы: резидентные, постоянно находящиеся в оперативной памяти, и транзитные, загружаемые в оперативную память только на время выполнения своих функций;
- реализация двух режимов работы вычислительной системы: привилегированного режима (режима ядра – Kernelmode), или режима супервизора (supervisormode), и пользовательского режима (usermode), или режима задачи (taskmode);
- ограничение функций ядра (а следовательно, и количества модулей ядра) до минимального количества необходимых самых важных функций.

Первые ОС разрабатывались как монолитные системы без четко выраженной структуры (рис. 1.2).

Для построения монолитной системы необходимо скомпилировать все отдельные процедуры, а затем связать их вместе в единый объектный файл с помощью компоновщика (примерами могут служить ранние версии ядра UNIX или NovellNetWare). Каждая процедура видит любую другую процедуру (в отличие от структуры, содержащей модули, в которой большая часть информации является локальной для модуля, и процедуры модуля можно вызвать только через специально определенные точки входа).

Однако даже такие монолитные системы могут быть немного структурированными. При обращении к системным вызовам, поддерживаемым ОС, параметры помещаются в строго определенные места, такие как регистры или стек, а затем выполняется специальная команда прерывания, известная как вызов ядра или вызов супервизора. Эта команда переключает машину из режима пользователя в режим ядра, называемый также режимом супервизора, и передает управление ОС. Затем ОС проверяет параметры вызова, для того чтобы определить, какой системный вызов должен быть выполнен. После этого ОС индексирует таблицу, содержащую ссылки на процедуры, и вызывает соответствующую процедуру.

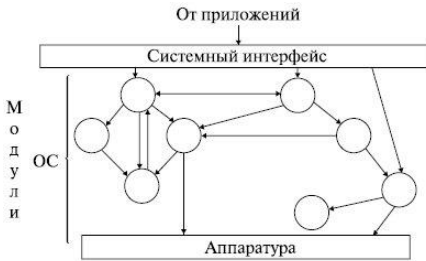


Рис. 1.2. Монолитная архитектура

Такая организация ОС предполагает следующую структуру :

- главная программа, которая вызывает требуемые сервисные процедуры;
- набор сервисных процедур, реализующих системные вызовы;
- набор утилит, обслуживающих сервисные процедуры.

Лекция 3

Структурированная архитектура.

В этой модели для каждого системного вызова имеется одна сервисная процедура. Утилиты выполняют функции, которые нужны нескольким сервисным процедурам. Это деление процедур на три слоя показано на рис. 1.3.

Классической считается архитектура ОС, основанная на концепции иерархической многоуровневой машины, привилегированном ядре и пользовательском режиме работы транзитных модулей. Модули ядра выполняют базовые функции ОС: управление процессами, памятью, устройствами ввода-вывода и т.п. Ядро составляет сердцевину ОС, без которой она является полностью неработоспособной и не может выполнить ни одну из своих функций. В ядре решаются внутрисистемные задачи организации вычислительного процесса, недоступные для приложения.

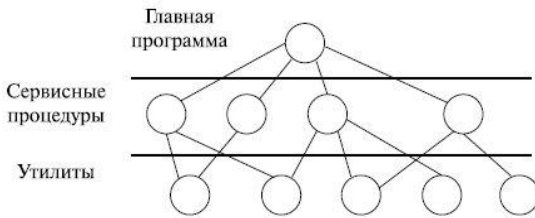


Рис. 1.3. Структурированная архитектура

Особый класс функций ядра служит для поддержки приложений, создавая для них так называемую прикладную программную среду. Приложения могут обращаться к ядру с запросами – системными вызовами – для выполнения тех или иных действий, например, открытие и чтение файла, получение системного времени, вывода информации на дисплей и т.д. Функции ядра, которые могут вызываться приложениями, образуют интерфейс прикладного программирования – API (Application Programming Interface).

Для обеспечения высокой скорости работы ОС модули ядра (по крайней мере, большая их часть) являются резидентными и работают в привилегированном режиме (Kernel mode). Этот режим, во-первых, должен обезопасить работу самой ОС от вмешательства приложений, и, во-вторых, должен обеспечить возможность работы модулей ядра с полным набором

машинных инструкций, позволяющих собственно ядру выполнять управление ресурсами компьютера, в частности, переключение процессора с задачи на задачу, управлением устройствами ввода-вывода, распределением и защитой памяти и др.

Остальные модули ОС выполняют не столь важные функции, как ядро, и являются транзитными. Например, это могут быть программы архивирования данных, дефрагментации диска, сжатия дисков, очистки дисков и т.п.

Вспомогательные модули обычно подразделяются на группы:

- утилиты – программы, выполняющие отдельные задачи управления и сопровождения вычислительной системы;
- системные обрабатывающие программы – текстовые и графические редакторы (Paint, Imaging в Windows 2000), компиляторы и др.;
- программы предоставления пользователю дополнительных услуг (специальный вариант пользовательского интерфейса, калькулятор, игры, средства мультимедиа Windows 2000);
- библиотеки процедур различного назначения, упрощения разработки приложений, например, библиотека функций ввода-вывода, библиотека математических функций и т.п.

Эти модули ОС оформляются как обычные приложения, обращаются к функциям ядра посредством системных вызовов и выполняются в пользовательском режиме (usermode). В этом режиме запрещается выполнение некоторых команд, которые связаны с функциями ядра ОС (управление ресурсами, распределение и защита памяти и т.п.).

В концепции многоуровневой (многослойной) иерархической машины структура ОС также представляется рядом слоев. При такой организации каждый слой обслуживает вышележащий слой, выполняя для него некоторый набор функций, которые образуют межслойный интерфейс. На основе этих функций следующий верхний по иерархии слой строит свои функции – более сложные и более мощные и т.д. Такая организация системы существенно упрощает ее разработку, т.к. позволяет сначала "сверху вниз" определить функции слоев и межслойные интерфейсы, а при детальной реализации, двигаясь "снизу вверх", – наращивать мощность функции слоев. Кроме того, модули каждого слоя можно изменять без необходимости изменений в других слоях (но не меняя межслойных интерфейсов!).

Многослойная структура ядра ОС может быть представлена, например, вариантом, показанным на рис. 1.4.

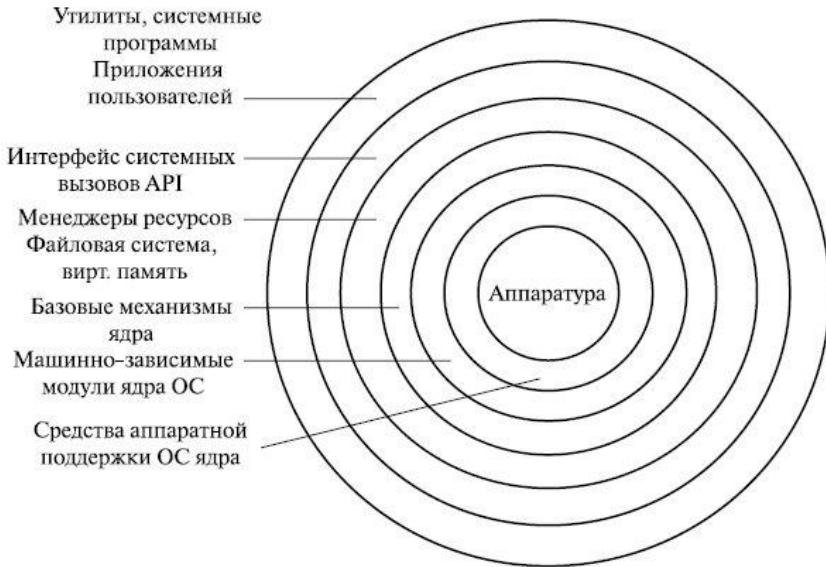


Рис. 1.4. Многослойная структура ОС

В данной схеме выделены следующие слои.

1. Средства аппаратной поддержки ОС. Значительная часть функций ОС может выполняться аппаратными средствами. Чисто программные ОС сейчас не существуют. Как правило, в современных системах всегда есть средства аппаратной поддержки ОС, которые прямо участвуют в организации вычислительных процессов. К ним относятся: система прерываний, средства поддержки привилегированного режима, средства поддержки виртуальной памяти, системный таймер, средства переключения контекстов процессов (информация о состоянии процесса в момент его приостановки), средства защиты памяти и др.
2. Машинно-зависимые модули ОС. Этот слой образует модули, в которых отражается специфика аппаратной платформы компьютера. Назначение этого слоя – "экранирование" вышележащих слоев ОС от особенностей аппаратуры (например, Windows 2000 – это слой HAL (Hardware Abstraction Layer), уровень аппаратных абстракций).

3. Базовые механизмы ядра. Этот слой модулей выполняет наиболее примитивные операции ядра: программное переключение контекстов процессов, диспетчерскую прерываний, перемещение страниц между основной памятью и диском и т.п. Модули этого слоя не принимают решений о распределении ресурсов, а только обрабатывают решения, принятые модулями вышележащих уровней. Поэтому их часто называют исполнительными механизмами для модулей верхних слоев ОС.
4. Менеджеры ресурсов. Модули этого слоя выполняют стратегические задачи по управлению ресурсами вычислительной системы. Это менеджеры (диспетчеры) процессов ввода-вывода, оперативной памяти и файловой системы. Каждый менеджер ведет учет свободных и используемых ресурсов и планирует их распределение в соответствии запросами приложений.
5. Интерфейс системных вызовов. Это верхний слой ядра ОС, взаимодействующий с приложениями и системными утилитами, он образует прикладной программный интерфейс ОС. Функции API, обслуживающие системные вызовы, предоставляют доступ к ресурсам системы в удобной компактной форме, без указания деталей их физического расположения.

Повышение устойчивости ОС обеспечивается переходом ядра в привилегированный режим. При этом происходит некоторое замедление выполнения системных вызовов. Системный вызов привилегированного ядра инициирует переключение процессора из пользовательского режима в привилегированный, а при возврате к приложению – обратное переключение. За счет этого возникает дополнительная задержка в обработке системного вызова (рис. 1.5). Однако такое решение стало классическим и используется во многих ОС (UNIX, VAX, VMS, IBM OS/390, OS/2 и др.).



Рис. 1.5. Обработка системного вызова

Многослойная классическая многоуровневая архитектура ОС не лишена своих проблем. Дело в том, что значительные изменения одного из уровней могут иметь трудно предвидимое влияние на смежные уровни. Кроме того, многочисленные взаимодействия между соседними уровнями усложняют обеспечение безопасности. Поэтому, как альтернатива классическому варианту архитектуры ОС, часто используется микроядерная архитектура ОС.

Суть этой архитектуры состоит в следующем. В привилегированном режиме остается работать только очень небольшая часть ОС, называемая микроядром. Микроядро защищено от остальных частей ОС и приложений. В его состав входят машинно-зависимые модули, а также модули, выполняющие базовые механизмы обычного ядра. Все остальные более высокоуровневые функции ядра оформляются как модули, работающие в пользовательском режиме. Так, менеджеры ресурсов, являющиеся неотъемлемой частью обычного ядра, становятся "периферийными" модулями, работающими в пользовательском режиме. Таким образом, в архитектуре с микроядром традиционное расположение уровней по вертикали заменяется горизонтальным. Это можно представить, как показано на рис. 1.6.

Внешние по отношению к микроядру компоненты ОС реализуются как обслуживающие процессы. Между собой они взаимодействуют как равноправные партнеры с помощью обмена сообщениями, которые передаются через микроядро. Поскольку назначением этих компонентов ОС

является обслуживание запросов приложений пользователей, утилит и системных обрабатывающих программ, менеджеры ресурсов, вынесенные в пользовательский режим, называются серверами ОС, т.е. модулями, основным назначением которых является обслуживание запросов локальных приложений и других модулей ОС.



Рис. 1.6. Переход к микроядерной архитектуре

Схематично механизм обращений к функциям ОС, оформленным в виде серверов, выглядит, как показано на рис. 1.7.



Рис. 1.7. Клиент-серверная архитектура

Схема смены режимов при выполнении системного вызова в ОС с микроядерной архитектурой выглядит, как показано на рис. 1.8. Из рисунка ясно, что выполнение системного вызова сопровождается четырьмя переключениями режимов (4 t), в то время как в классической архитектуре – двумя. Следовательно, производительность ОС с микроядерной архитектурой при прочих равных условиях будет ниже, чем у ОС с классическим ядром.



Рис. 1.8. Обработка системного вызова в микроядерной архитектуре

В то же время признаны следующие достоинства микроядерной архитектуры :

- единообразные интерфейсы;
- простота расширяемости;
- высокая гибкость;
- возможность переносимости;
- высокая надежность;
- поддержка распределенных систем;
- поддержка объектно-ориентированных ОС.

По многим источникам вопрос масштабов потери производительности в микроядерных ОС является спорным. Многогое зависит от размеров и функциональных возможностей микроядра. Избирательное увеличение функциональности микроядра приводит к снижению количества переключений между режимами системы, а также переключений адресных пространств процессов.

Может быть, это покажется парадоксальным, но есть и такой подход к микроядерной ОС, как уменьшение микроядра.

Для возможности представления о размерах микроядер операционных систем в ряде источников приводятся такие данные:

- типичное микроядро первого поколения – 300 Кбайт кода и 140 интерфейсов системных вызовов;
- микроядро ОС L4 (второе поколение) – 12 Кбайт кода и 7 интерфейсов системных вызовов.

В современных операционных системах различают следующие виды ядер.

1. Наноядро (НЯ). Крайне упрощённое и минимальное ядро, выполняет лишь одну задачу – обработку аппаратных прерываний, генерируемых устройствами компьютера. После обработки посылает информацию о результатах обработки вышележащему программному обеспечению. НЯ используются для виртуализации аппаратного обеспечения реальных компьютеров или для реализации механизма гипервизора.
2. Микроядро (МЯ) предоставляет только элементарные функции управления процессами и минимальный набор абстракций для работы с оборудованием. Большая часть работы осуществляется с помощью специальных пользовательских процессов, называемых сервисами. В микроядерной операционной системе можно, не прерывая ее работы, загружать и выгружать новые драйверы, файловые системы и т. д. Микроядерными являются ядра ОС Minix и GNU Hurd и ядро систем семейства BSD. Классическим примером микроядерной системы является Symbian OS. Это пример распространенной и отработанной микроядерной (а начиная с версии Symbian OS v8.1, и наноядерной) операционной системы.
3. Экзоядро (ЭЯ) предоставляет лишь набор сервисов для взаимодействия между приложениями, а также необходимый минимум функций, связанных с защитой: выделение и высвобождение ресурсов, контроль прав доступа и т. д. ЭЯ не занимается предоставлением абстракций для физических ресурсов – эти функции выносятся в библиотеку пользовательского уровня (так называемую libOS). В отличие от микроядра ОС, базирующиеся на ЭЯ, обеспечивают большую эффективность за счет отсутствия необходимости в переключении между процессами при каждом обращении к оборудованию.
4. Монолитное ядро (МНЯ) предоставляет широкий набор абстракций оборудования. Все части ядра работают в одном адресном пространстве.

МнЯ требуют перекомпиляции при изменении состава оборудования. Компоненты операционной системы являются не самостоятельными модулями, а составными частями одной программы. МнЯ более производительны, чем микроядро, поскольку работает как один большой процесс. МнЯ является большинство Unix-систем и Linux. Монолитность ядер усложняет отладку, понимание кода ядра, добавление новых функций и возможностей, удаление ненужного, унаследованного от предыдущих версий кода. "Разбухание" кода монолитных ядер также повышает требования к объёму оперативной памяти.

5. Модульное ядро (Мод. Я) – современная, усовершенствованная модификация архитектуры МЯ. В отличие от "классических" МнЯ, модульные ядра не требуют полной перекомпиляции ядра при изменении состава аппаратного обеспечения компьютера. Вместо этого они предоставляют тот или иной механизм подгрузки модулей, поддерживающих то или иное аппаратное обеспечение (например, драйверов). Подгрузка модулей может быть как динамической, так и статической (при перезагрузке ОС после переконфигурирования системы). Мод. Я удобнее для разработки, чем традиционные монолитные ядра. Они предоставляют программный интерфейс (API) для связывания модулей с ядром, для обеспечения динамической подгрузки и выгрузки модулей. Не все части ядра могут быть сделаны модулями. Некоторые части ядра всегда обязаны присутствовать в оперативной памяти и должны быть жёстко "вшиты" в ядро.
6. Гибридное ядро (ГЯ) – модифицированные микроядра, позволяющие для ускорения работы запускать "несущественные" части в пространстве ядра. Имеют "гибридные" достоинства и недостатки. Примером смешанного подхода может служить возможность запуска операционной системы с монолитным ядром под управлением микроядра. Так устроены 4.4BSD и MkLinux, основанные на микроядре Mach. Микроядро обеспечивает управление виртуальной памятью и работу низкоуровневых драйверов. Все остальные функции, в том числе взаимодействие с прикладными программами, осуществляются монолитным ядром. Данный подход сформировался в результате попыток использовать преимущества микроядерной архитектуры, сохраняя по возможности хорошо отлаженный код монолитного ядра.
7. Наиболее тесно элементы микроядерной архитектуры и элементы монолитного ядра переплетены в ядре Windows NT. Хотя Windows NT часто называют микроядерной операционной системой, это не совсем так. Микроядро NT слишком велико (более 1 Мбайт), чтобы носить приставку "микро". Компоненты ядра Windows NT

располагаются в вытесняемой памяти и взаимодействуют друг с другом путем передачи сообщений, как и положено в микроядерных операционных системах. В то же время все компоненты ядра работают в одном адресном пространстве и активно используют общие структуры данных, что свойственно операционным системам с монолитным ядром.

Тема 1.3. Общие сведения о процессах и потоках

Лекция 4

Модель процесса. Создание процесса. Завершение процесса.

Процессом, по существу, называют программу в момент выполнения.

Модель процесса. В этой модели все функционирующее на компьютере программное обеспечение, иногда включая собственно операционную систему, организовано в виде набора последовательных процессов, или, для краткости, просто процессов. Процессом является выполняемая программа, включая текущие значения счетчика команд, регистров и переменных. С позиций данной абстрактной модели, у каждого процесса есть собственный виртуальный центральный процессор. На самом деле, разумеется, реальный процессор переключается с процесса на процесс, но для лучшего понимания системы значительно проще рассматривать набор процессов, идущих параллельно (псевдопараллельно), чем пытаться представить себе процессор, переключающийся от программы к программе. Это переключение и называется многозадачностью или мультипрограммированием. Поскольку процессор переключается между программами, скорость, с которой процессор производит свои вычисления, будет непостоянной и, возможно, даже будет отличной при каждом новом запуске процесса. Поэтому не следует программировать процессы, исходя из каких-либо жестко заданных временных предположений. Различие между процессом и программой трудноуловимо и, тем не менее, имеет принципиальное значение. Воспользуемся следующей аналогией: представьте себе программиста, разбирающегося в кулинарии и пекущего торт на день рождения своей дочери. В его распоряжении есть рецепт торта, кухня, оборудованная всем необходимым, и ингредиенты для торта: мука, яйца, сахар, ванилин и т. п. Согласно этой аналогии, рецепт — это программа (то есть алгоритм, записанный в заданном виде), программист исполняет роль процессора, а ингредиенты торта являются входными данными. Процессом является следующая последовательность действий: программист читает рецепт, смешивает продукты и печет торт. Теперь представьте, что на кухню прибегает плачущий сын программиста и кричит, что его ужалила пчела. Программист отмечает, на чем он остановился (сохраняет текущее состояние процесса), находит справочник по оказанию первой помощи и действует в соответствии с инструкцией. Таким образом, наш процессор переключился с одного процесса (выпечка торта) на другой, с большим приоритетом (оказание первой помощи), и у каждого процесса есть своя программа (рецепт торта и справочник по оказанию первой помощи). После проведения всех необходимых процедур по борьбе с укусом пчелы программист возвращается к тарту, продолжая с той операции, на которой он прервался. Мы привели эту

аналогию с целью показать, что процесс — это активность некоторого рода. У него есть программа, входные и выходные данные, а также состояние. Один процессор может переключаться между различными процессами, используя некий алгоритм планирования для определения момента переключения от одного процесса к другому.

Создание процесса. Операционной системе необходим способ, позволяющий удостовериться в наличии всех необходимых процессов. В простейших системах, а также системах, разработанных для выполнения одного-единственного приложения, можно реализовать такую ситуацию, в которой все процессы, которые когда-либо могут понадобиться, присутствуют в системе при ее загрузке. В универсальных системах необходим способ создания и прерывания процессов по мере необходимости. В этом разделе мы рассмотрим некоторые из возможных способов решения этой проблемы. Ниже перечислены четыре основных события, приводящие к созданию процессов.

- Инициализация системы.
- Выполнение изданного работающим процессом системного запроса на создание процесса.
- Запрос пользователя на создание процесса.
- Инициирование пакетного задания.

Обычно при загрузке операционной системы создаются несколько процессов. Некоторые из них являются высокоприоритетными процессами, то есть обеспечивающими взаимодействие с пользователем и выполняющими заданную работу. Остальные процессы являются фоновыми, они не связаны с конкретными пользователями, но выполняют особые функции. Например, один фоновый процесс может быть предназначен для обработки входящей на компьютер почты, активизируясь только по мере появления писем. Другой фоновый процесс может обрабатывать запросы к web-страницам, расположенным на компьютере, и активизироваться для обслуживания полученного запроса. Фоновые процессы, связанные с электронной почтой, web-страницами, новостями, выводом на печать и т. п., называются демонами.

Процессы могут создаваться не только в момент загрузки системы, но и позже. Например, новый процесс (или несколько) может быть создан по просьбе текущего процесса. В интерактивных системах пользователь может запустить программу, набрав на клавиатуре команду или дважды щелкнув на

значке программы. В обоих случаях результатом будет создание нового процесса и запуск в нем программы.

С технической точки зрения во всех перечисленных случаях новый процесс формируется одинаково: текущий процесс выполняет системный запрос на создание нового процесса. В роли текущего процесса может выступать процесс, запущенный пользователем, системный процесс, инициированный клавиатурой или мышью, а также процесс, управляющий пакетами. В любом случае этот процесс всего лишь выполняет системный запрос на создание нового процесса. Системный запрос заставляет операционную систему создать новый процесс, а также прямо или косвенно содержит информацию о программе, которую нужно запустить в этом процессе.

В UNIX существует только один системный запрос, направленный на создание нового процесса: `fork`. Этот запрос создает дубликат вызываемого процесса. После выполнения запроса `fork` двум процессам — родительскому и дочернему — соответствуют одинаковые образы памяти, строки окружения и одни и те же открытые файлы. Обычно дочерний процесс выполняет системный вызов `execve` для изменения своего образа памяти и запуска новой программы. Так, когда пользователь набирает на клавиатуре команду `sort`, оболочка создает путем ветвления дочерний процесс, который и выполняет программу `sort`. Смысл этого двухступенчатого процесса заключается в том, что дочерний процесс успевает обработать описания файлов после `fork`, но до `execve`, чтобы выполнить перенаправление стандартных устройств ввода и вывода и потока сообщений об ошибках. И в UNIX, и в Windows после создания нового процесса родительский и дочерний процессы имеют собственные различные адресные пространства. При изменении любым процессом слова в адресном пространстве это изменение незаметно для других процессов. В UNIX начальное адресное пространство дочернего процесса является копией родительского, но сами адресные пространства различны, и перезаписываемая память совместно не используется (некоторые приложения UNIX совместно используют текст программы, поскольку его нельзя модифицировать). В то же время созданный процесс может использовать совместно с родительским процессом некоторые другие ресурсы, например открытые файлы. В Windows адресные пространства родительского и дочернего процессов отличаются с самого начала.

Завершение процесса. После того как процесс создан, он начинает выполнять свою работу. Но ничто не длится вечно, даже процесс — рано или поздно он завершится, чаще всего благодаря одному из следующих событий:

- Обычный выход (преднамеренно).
- Выход по ошибке (преднамеренно).
- Выход по неисправимой ошибке (непреднамеренно).
- Уничтожение другим процессом (непреднамеренно).

В основном процессы завершаются по мере выполнения своей работы. После окончания компиляции программы компилятор выполняет системный запрос, чтобы сообщить операционной системе об окончании работы. Второй причиной завершения процесса может стать неустранимая ошибка. Например, если пользователь набрал на клавиатуре команду

ее foo.c для компиляции программы foo.c, а соответствующего файла не существует, компилятор просто закончит работу. Интерактивные процессы, рассчитанные на работу с экраном, обычно не завершают работу при получении неверных параметров, вместо этого выводя на экран диалоговое окно и прося пользователя ввести правильные параметры.

Третьей причиной завершения процесса является ошибка, вызванная самим процессом, чаще всего связанная с ошибкой в программе. В качестве примера можно привести выполнение недопустимой команды, обращение к несуществующей области памяти и деление на ноль.

Четвертой причиной завершения процесса может служить выполнение другим процессом системного запроса на уничтожение процесса.

Иерархия процессов. В некоторых системах родительский и дочерний процессы остаются связанными между собой определенным образом. Дочерний процесс также может, в свою очередь, создавать процессы, формируя иерархию процессов. Следует отметить, что у процесса может быть лишь один родитель и сколько угодно «детей». В UNIX процесс, все его «дети» и дальнейшие потомки образуют группу процессов. Сигнал, посылаемый пользователем с клавиатуры, доставляется всем членам группы, взаимодействующим с клавиатурой в данный момент (обычно это все активные процессы, созданные в текущем окне). Каждый из процессов может перехватить сигнал, игнорировать его или выполнить другое действие, предусмотренное по умолчанию. В Windows не существует понятия иерархии процессов, и все процессы равноправны. Единственное, в чем проявляется что-то вроде иерархии процессов — создание процесса, в котором родительский процесс получает специальный маркер (так называемый дескриптор), позволяющий контролировать дочерний процесс. Но маркер можно передать другому процессу, нарушая иерархию. В UNIX это невозможно.

Лекция 5

Передача информации от одного процесса другому

Передача может осуществляться несколькими способами:

- Разделяемая память
- **Каналы** (трубы), это псевдофайл, в который один процесс пишет, а другой читает.
- **Сокеты** - поддерживаемый ядром механизм, скрывающий особенности среды и позволяющий единообразно взаимодействовать процессам, как на одном компьютере, так и в сети.
- Почтовые ящики (только в Windows), однонаправленные, возможность широковещательной рассылки.
- Вызов удаленной процедуры, процесс **A** может вызвать процедуру в процессе **B**, и получить обратно данные.

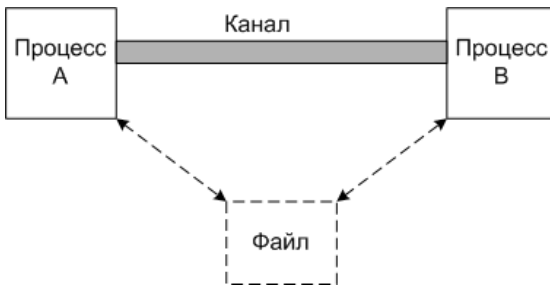


Схема для канала

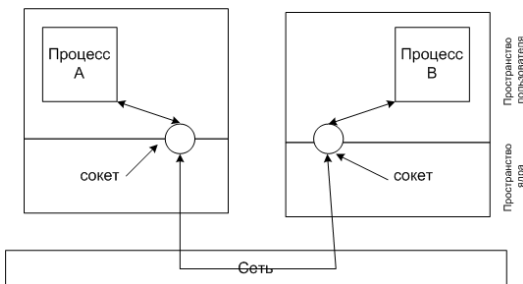


Схема для сокетов

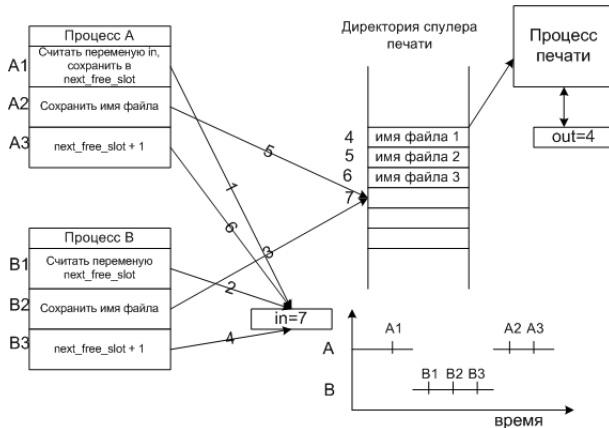
Состояние состязания

Состояние состязания - ситуация когда несколько процессов считывают или записывают данные (в память или файл) одновременно.

Рассмотрим пример, когда два процесса пытаются распечатать файл. Для этого им нужно поместить имя файла в спулер печати, в свободный сегмент.

in - переменная указывающая на следующий свободный сегмент

out - переменная указывающая на следующее имя файла для печати



Пример состязания

Распишем события по пунктам.

1. Процесс **A** считывает переменную **in** (равную 7), и сохраняет ее в своей переменной **next_free_slot**.
2. Происходит прерывание по таймеру, и процессор переключается на процесс **B**.
3. Процесс **B** считывает переменную **in** (равную 7), и сохраняет ее в своей переменной **next_free_slot**.
4. Процесс **B** сохраняет имя файла в сегменте 7.

5. Процесс **В** увеличивает переменную `next_free_slot` на единицу (`next_free_slot+1`), и заменяет значение `in` на 8.
6. Управление переходит процессу **А**, и продолжает с того места на котором остановился.
7. Процесс **А** сохраняет имя файла в сегменте 7, затирая имя файла процесса **В**.
8. Процесс **А** увеличивает переменную `next_free_slot` на единицу (`next_free_slot+1`), и заменяет значение `in` на 8.

Как видно из этой ситуации, файл процесса **В** не будет напечатан.

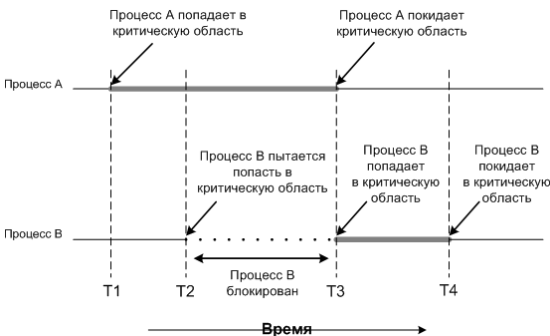
Критические области

Критическая область - часть программы, в которой есть обращение к совместно используемым данным.

Условия избегания состязания и эффективной работы процессов:

1. Два процесса не должны одновременно находиться в критических областях.
2. Процесс, находящийся вне критической области, не может блокировать другие процессы.
3. Невозможна ситуация, когда процесс вечно ждет (зависает) попадания в критическую область.

Пример:



Взаимное исключение с использованием критических областей

Взаимное исключение с активным ожиданием

Рассмотрим методы взаимного исключения

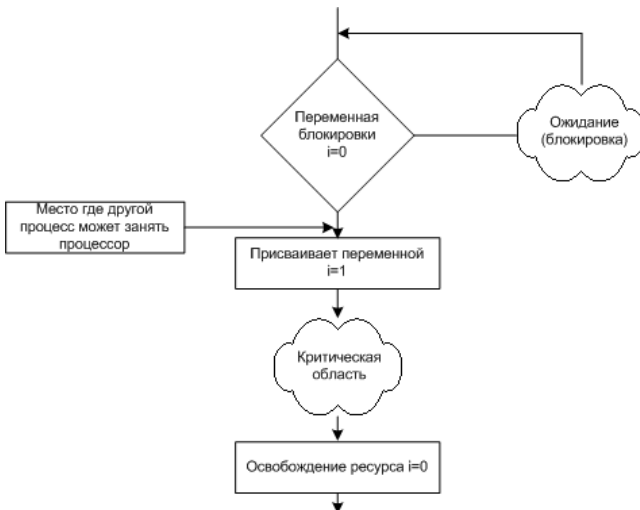
Запрещение прерываний

Заключается в запрещении всех прерываний при входе процесса в критическую область.

Недостаток этого метода в том, что если произойдет сбой процесса, то он не сможет снять запрет на прерывания.

Переменные блокировки

Вводится понятие переменной блокировки, т.е. если значение этой переменной равно, например 1, то ресурс занят другим процессом, и второй процесс переходит в режим ожидания (блокируется) до тех пор, пока переменная не примет значение 0.



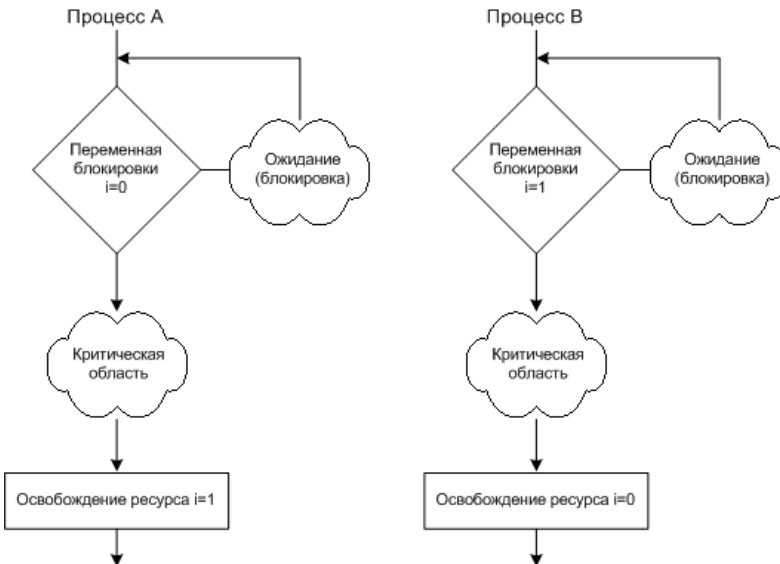
метод блокирующих переменных

Проблема, как и с процессом печати, после того как первый процесс считает 0, второй может занять процессор и тоже считать 0. Заблокированный

процесс находится в режиме **активного ожидания**, постоянно проверяя, не изменилась ли переменная блокировки.

Строгое чередование

В этой модели, процессы могут выполняться строго по очереди, используя переменную.



Строгое чередование

Недостатки метода:

- Заблокированный процесс постоянно находится в цикле, проверяя, не изменилась ли переменная.
- Противоречит третьему условию, когда процесс, находящийся вне критической области, может блокировать другие процессы.

Существуют еще алгоритмы с активным ожиданием (**алгоритм Петерсона, команда TSL**), но у всех них есть общий недостаток - расходуется бесцельно время процессора на циклы проверки изменения переменной.

Тема 1.4. Взаимодействие и планирование процессов.

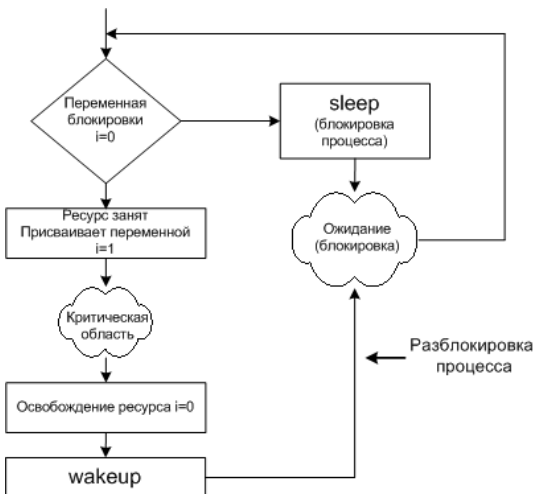
Лекция 6

Взаимодействие и планирование процессов.

Ситуации, когда приходится процессам взаимодействовать:

- Передача информации от одного процесса другому
- Контроль над деятельностью процессов (например: когда они борются за один ресурс)
- Согласование действий процессов (например: когда один процесс поставляет данные, а другой их выводит на печать. Если согласованности не будет, то второй процесс может начать печать раньше, чем поступят данные).

Два вторых случая относятся и к потокам. В первом случае у потоков нет проблем, т.к. они используют общее адресное пространство.



Применение примитивов

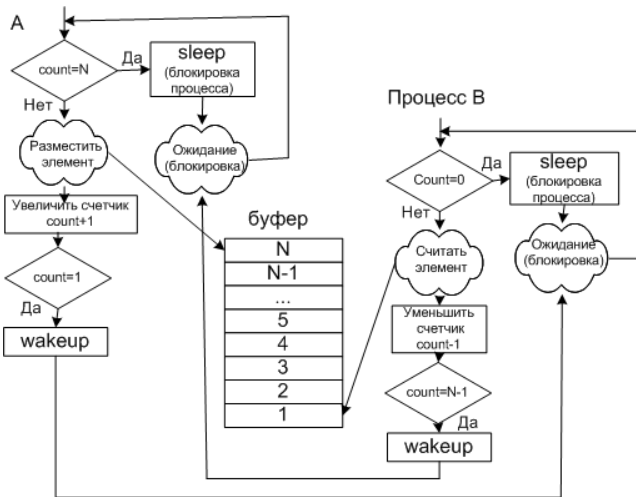
Основное преимущество - это отсутствие активного ожидания..

Проблема заключается в следующем, если спулер пуст, то wakeup срабатывает в пустую.

Проблема переполненного буфера (проблема производителя и потребителя)

Рассмотрим два процесса, которые совместно используют буфер ограниченного размера, один процесс пишет в буфер, другой считывает данные.

Чтобы первый процесс не писал, когда буфер полный, а второй не считывал, когда он пуст, вводится переменная count для подсчета количества элементов в буфере.



Проблема переполненного буфера

В этой ситуации оба процесса могут попасть в состояние ожидания, если пропадет сигнал активации.

Алгоритм такой ситуации:

1. Процесс **В**, считал count=0 (заблокироваться он еще не успел)
2. Планировщик передал управление процессу **А**
3. Процесс **А**, выполнил все вплоть до wakeup, пытаясь разблокировать процесс **В** (но он не заблокирован, wakeup срабатывает впустую)
4. Планировщик передал управление процессу **В**
5. И он заблокировался, и больше сигнала на разблокировку не получит
6. Процесс **А** в конце концов заполнит буфер и заблокируется, но сигнала на разблокировку не получит.

Семафоры

Семафоры - переменные для подсчета сигналов запуска, сохраненных на будущее.

Были предложены две операции **down** и **up** (аналоги sleep и wakeup).

Прежде чем заблокировать процесс, down проверяет семафор, если он равен нулю, то он блокирует процесс, если нет, то процесс снова становится активным, и уменьшает семафор на единицу.

up увеличит значение семафора на 1 или разблокирует процесс находящийся в ожидании..

down уменьшает значение семафора на 1 или блокирует процесс, если семафор =0.

down и up выполняются как **элементарное действие**, т.е. процесс не может быть блокирован во время выполнения этих операций. Значит, у операционной системы должен быть запрет на все прерывания, и перевод процесса в режим ожидания.

Решение проблемы переполненного буфера с помощью семафора

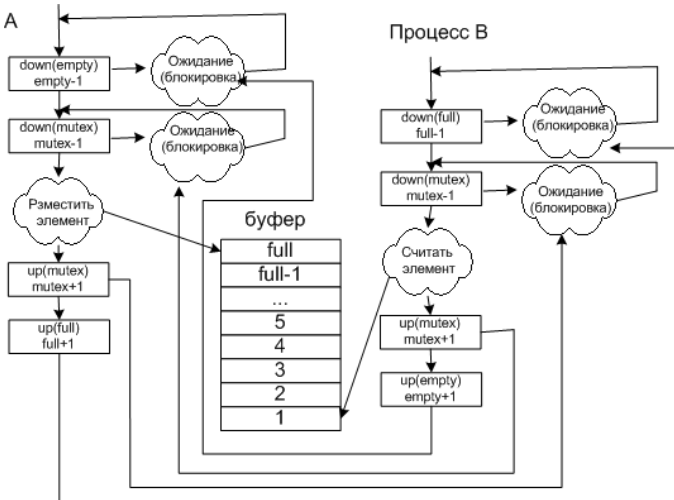
Применим три семафора:

full - подсчет заполненных сегментов (в начале = 0)

empty - подсчет пустых сегментов (в начале = количеству сегментов)

mutex - для исключения одновременного доступа к буферу двух процессов. (в начале = 1)

Мьютекс упрощенная версия семафора, он управляет доступом к ресурсу. Показывает, блокирован или нет ресурс.



Решение проблемы переполненного буфера с помощью семафора

Применение семафоров для устройств ввода/вывода

Для устройств ввода/вывода семафор выставляется равный нулю. После запуска управляющего процесса выполняется down, и т.к. семафор равен нулю, процесс блокируется. Когда нужно активизировать процесс управления, выполняется up.

Примитивы взаимодействия процессов

Вводится понятия двух примитивов.

sleep - системный запрос, в результате которого вызывающий процесс блокируется, пока его не запустит другой процесс.

wakeup - системный запрос, в результате которого блокированный процесс будет запущен.

Планирование процессов

Планирование заданий используется в качестве долгосрочного планирования процессов. Оно отвечает за порождение новых процессов в системе, определяя ее степень мультипрограммирования, т. е. количество процессов, одновременно находящихся в ней. Если степень мультипрограммирования системы поддерживается постоянной, т. е. среднее количество процессов в компьютере не меняется, то новые процессы могут появляться только после завершения ранее загруженных. Поэтому долгосрочное планирование осуществляется достаточно редко, между появлением новых процессов могут проходить минуты и даже десятки минут. Решение о выборе для запуска того или иного процесса оказывает влияние на функционирование вычислительной системы на протяжении достаточно длительного времени. Отсюда и название этого уровня планирования – долгосрочное. В некоторых операционных системах долгосрочное планирование сведено к минимуму или отсутствует вовсе. Так, например, во многих интерактивных системах разделения времени порождение процесса происходит сразу после появления соответствующего запроса. Поддержание разумной степени мультипрограммирования осуществляется за счет ограничения количества пользователей, которые могут работать в системе, и особенностей человеческой психологии. Если между нажатием на клавишу и появлением символа на экране проходит 20–30 секунд, то многие пользователи предпочтут прекратить работу и продолжить ее, когда система будет менее загружена.

Планирование использования процессора применяется в качестве краткосрочного планирования процессов. Оно проводится, к примеру, при обращении исполняющегося процесса к устройствам ввода-вывода или просто по завершении определенного интервала времени. Поэтому краткосрочное планирование осуществляется, как правило, не реже одного раза в 100 миллисекунд. Выбор нового процесса для исполнения оказывает влияние на функционирование системы до наступления очередного аналогичного события, т. е. в течение короткого промежутка времени, чем и обусловлено название этого уровня планирования – краткосрочное.

В некоторых вычислительных системах бывает выгодно для повышения производительности временно удалить какой-либо частично выполнившийся процесс из оперативной памяти на диск, а позже вернуть его обратно для

дальнейшего выполнения. Такая процедура в англоязычной литературе получила название *swapping*, что можно перевести на русский язык как «перекачка», хотя в специальной литературе оно употребляется без перевода – свопинг. Когда и какой из процессов нужно перекачать на диск и вернуть обратно, решается дополнительным промежуточным уровнем планирования процессов – среднесрочным.

Критерии планирования и требования к алгоритмам

Для каждого уровня планирования процессов можно предложить много различных алгоритмов. Выбор конкретного алгоритма определяется классом задач, решаемых вычислительной системой, и целями, которых мы хотим достичь, используя планирование. К числу таких целей можно отнести следующие:

- **Справедливость** – гарантировать каждому заданию или процессу определенную часть времени использования процессора в компьютерной системе, стараясь не допустить возникновения ситуации, когда процесс одного пользователя постоянно занимает процессор, в то время как процесс другого пользователя фактически не начинал выполняться.
- **Эффективность** – постараться занять процессор на все 100% рабочего времени, не позволяя ему простаивать в ожидании процессов, готовых к исполнению. В реальных вычислительных системах загрузка процессора колеблется от 40 до 90%.
- **Сокращение полного времени выполнения (turnaroundtime)** – обеспечить минимальное время между стартом процесса или постановкой задания в очередь для загрузки и его завершением.
- **Сокращение времени ожидания (waitingtime)** – сократить время, которое проводят процессы в состоянии готовности и задания в очереди для загрузки.
- **Сокращение времени отклика (responsetime)** – минимизировать время, которое требуется процессу в интерактивных системах для ответа на запрос пользователя.

Независимо от поставленных целей планирования желательно также, чтобы алгоритмы обладали следующими свойствами.

Были предсказуемыми. Одно и то же задание должно выполняться приблизительно за одно и то же время. Применение алгоритма планирования не должно приводить, к примеру, к извлечению корня квадратного из 4 за

сотые доли секунды при одном запуске и за несколько суток – при втором запуске.

Были связаны с минимальными накладными расходами. Если на каждые 100 миллисекунд, выделенные процессу для использования процессора, будет приходиться 200 миллисекунд на определение того, какой именно процесс получит процессор в свое распоряжение, и на переключение контекста, то такой алгоритм, очевидно, применять не стоит.

Равномерно загружали ресурсы вычислительной системы, отдавая предпочтение тем процессам, которые будут занимать малоиспользуемые ресурсы.

Обладали масштабируемостью, т. е. не сразу теряли работоспособность при увеличении нагрузки. Например, рост количества процессов в системе в два раза не должен приводить к увеличению полного времени выполнения процессов на порядок.

Многие из приведенных выше целей и свойств являются противоречивыми. Улучшая работу алгоритма с точки зрения одного критерия, мы ухудшаем ее с точки зрения другого. Приспосабливая алгоритм под один класс задач, мы тем самым дискриминируем задачи другого класса. «В одну телегу впрячь не можно коня и трепетную лань». Ничего не поделаешь. Такова жизнь.

Параметры планирования

Для осуществления поставленных целей разумные алгоритмы планирования должны опираться на какие-либо характеристики процессов в системе, заданий в очереди на загрузку, состояния самой вычислительной системы, иными словами, на параметры планирования. В этом разделе мы опишем ряд таких параметров, не претендуя на полноту изложения.

Все параметры планирования можно разбить на две большие группы: статические параметры и динамические параметры. Статические параметры не изменяются в ходе функционирования вычислительной системы, динамические же, напротив, подвержены постоянным изменениям.

К статическим параметрам вычислительной системы можно отнести предельные значения ее ресурсов (размер оперативной памяти, максимальное количество памяти на диске для осуществления свопинга, количество

подключенных устройств ввода-вывода и т. п.). Динамические параметры системы описывают количество свободных ресурсов на данный момент.

К статическим параметрам процессов относятся характеристики, как правило, присущие заданиям уже на этапе загрузки.

- Каким пользователем запущен процесс или сформировано задание.
- Насколько важной является поставленная задача, т. е. каков приоритет ее выполнения.
- Сколько процессорного времени запрошено пользователем для решения задачи.
- Каково соотношение процессорного времени и времени, необходимого для осуществления операций ввода-вывода.
- Какие ресурсы вычислительной системы (оперативная память, устройства ввода-вывода, специальные библиотеки и системные программы и т. д.) и в каком количестве необходимы заданию.

Алгоритмы долгосрочного планирования используют в своей работе статические и динамические параметры вычислительной системы и статические параметры процессов (динамические параметры процессов на этапе загрузки заданий еще не известны). Алгоритмы краткосрочного и среднесрочного планирования дополнительно учитывают и динамические характеристики процессов. Для среднесрочного планирования в качестве таких характеристик может использоваться следующая информация:

- сколько времени прошло с момента выгрузки процесса на диск или его загрузки в оперативную память;
- сколько оперативной памяти занимает процесс;
- сколько процессорного времени уже предоставлено процессу.

Тема 1.5. Управление памятью.

Лекция 7

Тема 1.5.1. Абстракция памяти. Виртуальная память.

Понятие виртуальной памяти

Разработчикам программного обеспечения часто приходится решать проблему размещения в памяти больших программ, размер которых превышает объем доступной оперативной памяти. Один из вариантов решения данной проблемы – организация структур с перекрытием – рассмотрен в предыдущей лекции. При этом предполагалось активное участие программиста в процессе формирования перекрывающихся частей программы. Развитие архитектуры компьютеров и расширение возможностей операционной системы по управлению памятью позволило переложить решение этой задачи на компьютер. Одним из главных достижений стало появление виртуальной памяти (virtualmemory). Впервые она была реализована в 1959 г. на компьютере "Атлас", разработанном в Манчестерском университете.

Суть концепции **виртуальной памяти** заключается в следующем. Информация, с которой работает активный процесс, должна располагаться в оперативной памяти. В схемах виртуальной памяти у процесса создается иллюзия того, что вся необходимая ему информация имеется в основной памяти. Для этого, во-первых, занимаемая процессом память разбивается на несколько частей, например страниц. Во-вторых, логический адрес (логическая страница), к которому обращается процесс, динамически транслируется в физический адрес (физическую страницу). И, наконец, в тех случаях, когда страница, к которой обращается процесс, не находится в физической памяти, нужно организовать ее подкачку с диска. Для контроля наличия страницы в памяти вводится специальный бит присутствия, входящий в состав атрибутов страницы в таблице страниц .

Таким образом, в наличии всех компонентов процесса в основной памяти необходимости нет. Важным следствием такой организации является то, что размер памяти, занимаемой процессом, может быть больше, чем размер оперативной памяти. Принцип локальности обеспечивает этой схеме нужную эффективность.

Возможность выполнения программы, находящейся в памяти лишь частично, имеет ряд вполне очевидных преимуществ.

- Программа не ограничена объемом физической памяти. Упрощается разработка программ, поскольку можно задействовать большие виртуальные пространства, не заботясь о размере используемой памяти.
- Поскольку появляется возможность частичного помещения программы (процесса) в память и гибкого перераспределения памяти между программами, можно разместить в памяти больше программ, что увеличивает загрузку процессора и пропускную способность системы.
- Объем ввода-вывода для выгрузки части программы на диск может быть меньше, чем в варианте классического свопинга, в итоге каждая программа будет работать быстрее.

Таким образом, возможность обеспечения (при поддержке операционной системы) для программы "видимости" практически неограниченной (характерный размер для 32-разрядных архитектур $2^{32} = 4$ Гбайт) адресуемой пользовательской памяти (логическое адресное пространство) при наличии основной памяти существенно меньших размеров (физическое адресное пространство) – очень важный аспект.

Но введение виртуальной памяти позволяет решать другую, не менее важную задачу – обеспечение контроля доступа к отдельным сегментам памяти и, в частности, защиту пользовательских программ друг от друга и защиту ОС от пользовательских программ. Каждый процесс работает со своими виртуальными адресами, трансляцию которых в физические выполняет аппаратура компьютера. Таким образом, пользовательский процесс лишен возможности напрямую обратиться к страницам основной памяти, занятым информацией, относящейся к другим процессам.

Например, 16-разрядный компьютер PDP-11/70 с 64 Кбайт логической памяти мог иметь до 2 Мбайт оперативной памяти. Операционная система этого компьютера тем не менее поддерживала виртуальную память, которая обеспечивала защиту и перераспределение основной памяти между пользовательскими процессами.

Напомним, что в системах с виртуальной памятью те адреса, которые генерирует программа (логические адреса), называются виртуальными, и они формируют виртуальное адресное пространство. Термин "**виртуальная память**" означает, что программист имеет дело с памятью, отличной от реальной, размер которой потенциально больше, чем размер оперативной памяти.

Хотя известны и чисто программные реализации виртуальной памяти, это направление получило наиболее широкое развитие после соответствующей аппаратной поддержки.

Следует отметить, что оборудование компьютера принимает участие в трансляции адреса практически во всех схемах управления памятью. Но в случае виртуальной памяти это становится более сложным вследствие разрывности отображения и многомерности логического адресного пространства. Может быть, наиболее существенным вкладом аппаратуры в реализацию описываемой схемы является автоматическая генерация исключительных ситуаций при отсутствии в памяти нужных страниц (pagefault).

Любая из трех ранее рассмотренных схем управления памятью – страничной, сегментной и сегментно-страничной – пригодна для организации виртуальной памяти. Чаще всего используется сегментно-страничная модель, которая является синтезом страничной модели и идеи сегментации. Причем для тех архитектур, в которых сегменты не поддерживаются аппаратно, их реализация – задача архитектурно-независимого компонента менеджера памяти.

Сегментная организация в чистом виде встречается редко.

Архитектурные средства поддержки виртуальной памяти

Очевидно, что невозможно создать полностью машинно-независимый компонент управления виртуальной памятью. С другой стороны, имеются существенные части программного обеспечения, связанного с управлением виртуальной памятью, для которых детали аппаратной реализации совершенно не важны. Одним из достижений современных ОС является грамотное и эффективное разделение средств управления виртуальной памятью на аппаратно-независимую и аппаратно-зависимую части. Коротко рассмотрим, что и каким образом входит в аппаратно-зависимую часть подсистемы управления виртуальной памятью. Компоненты аппаратно-независимой подсистемы будут рассмотрены в следующей лекции.

В самом распространенном случае необходимо отобразить большое виртуальное адресное пространство в физическое адресное пространство существенно меньшего размера. Пользовательский процесс или ОС должны иметь возможность осуществить запись по виртуальному адресу,

а задача ОС – сделать так, чтобы записанная информация оказалась в физической памяти (впоследствии при нехватке оперативной памяти она может быть вытеснена во внешнюю память). В случае виртуальной памяти система отображения адресных пространств помимо трансляции адресов должна предусматривать ведение таблиц, показывающих, какие области виртуальной памяти в данный момент находятся в физической памяти и где именно размещаются.

Страничная виртуальная память

Как и в случае простой **страничной организации**, страничная виртуальная память и физическая память представляются состоящими из наборов блоков или страниц одинакового размера. Виртуальные адреса делятся на страницы (page), соответствующие единицы в физической памяти образуют страничные кадры (pageframes), а в целом система поддержки страничной виртуальной памяти называется пейджингом (paging). Передача информации между памятью и диском всегда осуществляется целыми страницами.

После разбиения менеджером памяти виртуального адресного пространства на страницы виртуальный адрес преобразуется в упорядоченную пару (p, d) , где p – номер страницы в виртуальной памяти, а d – смещение в рамках страницы p , внутри которой размещается адресуемый элемент. Процесс может выполняться, если его текущая страница находится в оперативной памяти. Если текущей страницы в главной памяти нет, она должна быть переписана (подкачена) из внешней памяти. Поступившую страницу можно поместить в любой свободный страничный кадр.

Поскольку число виртуальных страниц велико, таблица страниц принимает специфический вид (см. раздел "Структура таблицы страниц"), структура записей становится более сложной, среди атрибутов страницы появляются биты присутствия, модификации и другие управляющие биты.

При отсутствии страницы в памяти в процессе выполнения команды возникает исключительная ситуация, называемая страничное нарушение (pagefault) или страничный отказ. Обработка страничного нарушения заключается в том, что выполнение команды прерывается, затребованная страница подкачивается из конкретного места вторичной памяти в свободный страничный кадр физической памяти и попытка выполнения команды повторяется. При отсутствии свободных страничных кадров на диск выгружается редко используемая страница. Проблемы

замещения страниц и обработки страничных нарушений рассматриваются в следующей лекции.

Для управления физической памятью ОС поддерживает структуру таблицы кадров. Она имеет одну запись на каждый физический кадр, показывающую его состояние.

В большинстве современных компьютеров со страничной организацией в основной памяти хранится лишь часть таблицы страниц, а быстрота доступа к элементам таблицы текущей виртуальной памяти достигается, как будет показано ниже, за счет использования сверхбыстродействующей памяти, размещенной в кэше процессора.

Сегментно-страничная организации виртуальной памяти

Как и в случае простой сегментации, в схемах виртуальной памяти сегмент – это линейная последовательность адресов, начинающаяся с 0. При организации виртуальной памяти размер сегмента может быть велик, например, может превышать размер оперативной памяти. Повторяя все ранее приведенные рассуждения о размещении в памяти больших программ, приходим к разбиению сегментов на страницы и необходимости поддержки своей таблицы страниц для каждого сегмента.

На практике, однако, появления в системе большого количества таблиц страниц стараются избежать, организовав неперекрывающиеся сегменты в одном виртуальном пространстве, для описания которого хватает одной таблицы страниц. Таким образом, одна таблица страниц отводится для всего процесса. Например, в популярных ОС Linux и Windows 2000 все сегменты процесса, а также область памяти ядра ограничены виртуальным адресным пространством объемом 4 Гбайт. При этом ядро ОС располагается по фиксированным виртуальным адресам вне зависимости от выполняемого процесса.

Ассоциативная память

Поиск номера кадра, соответствующего нужной странице, в многоуровневой таблице страниц требует нескольких обращений к основной памяти, поэтому занимает много времени. В некоторых случаях такая задержка недопустима. Проблема ускорения поиска решается на уровне архитектуры компьютера.

В соответствии со свойством локальности большинство программ в течение некоторого промежутка времени обращаются к небольшому количеству страниц, поэтому активно используется только небольшая часть таблицы страниц.

Естественное решение проблемы ускорения – снабдить компьютер аппаратным устройством для отображения виртуальных страниц в физические без обращения к таблице страниц, то есть иметь небольшую, быструю кэш-память, хранящую необходимую на данный момент часть таблицы страниц. Это устройство называется **ассоциативной памятью**, иногда также употребляют термин буфер поиска трансляции (translationlookasidebuffer – TLB).

Одна запись таблицы в ассоциативной памяти (один вход) содержит информацию об одной виртуальной странице: ее атрибуты и кадр, в котором она находится. Эти поля в точности соответствуют полям в таблице страниц.

Так как ассоциативная память содержит только некоторые из записей таблицы страниц, каждая запись в TLB должна включать поле с номером виртуальной страницы. Память называется ассоциативной, потому что в ней происходит одновременное сравнение номера отображаемой виртуальной страницы с соответствующим полем во всех строках этой небольшой таблицы. Поэтому данный вид памяти достаточно дорого стоит. В строке, поле виртуальной страницы которой совпало с искомым значением, находится номер страничного кадра. Обычное число записей в TLB от 8 до 4096. Рост количества записей в ассоциативной памяти должен осуществляться с учетом таких факторов, как размер кэша основной памяти и количества обращений к памяти при выполнении одной команды.

Рассмотрим функционирование менеджера памяти при наличии ассоциативной памяти.

В начале информация об отображении виртуальной страницы в физическую отыскивается в ассоциативной памяти. Если нужная запись найдена – все нормально, за исключением случаев нарушения привилегий, когда запрос на обращение к памяти отклоняется.

Если нужная запись в ассоциативной памяти отсутствует, отображение осуществляется через таблицу страниц. Происходит замена одной из записей в ассоциативной памяти найденной записью из таблицы страниц. Здесь мы

сталкиваемся с традиционной для любого кэша проблемой замещения (а именно какую из записей в кэше необходимо изменить). Конструкция ассоциативной памяти должна организовывать записи таким образом, чтобы можно было принять решение о том, какая из старых записей должна быть удалена при внесении новых.

Число удачных поисков номера страницы в ассоциативной памяти по отношению к общему числу поисков называется hit (совпадение) ratio (пропорция, отношение). Иногда также используется термин "процент попаданий в кэш". Таким образом, hitratio – часть ссылок, которая может быть сделана с использованием ассоциативной памяти. Обращение к одним и тем же страницам повышает hitratio. Чем больше hitratio, тем меньше среднее время доступа к данным, находящимся в оперативной памяти.

Предположим, например, что для определения адреса в случае кэш-промаха через таблицу страниц необходимо 100 нс, а для определения адреса в случае кэш-попадания через ассоциативную память – 20 нс. С 90% hitratio среднее время определения адреса – $0,9 \times 20 + 0,1 \times 100 = 28$ нс.

Вполне приемлемая производительность современных ОС доказывает эффективность использования ассоциативной памяти. Высокое значение вероятности нахождения данных в ассоциативной памяти связано с наличием у данных объективных свойств: пространственной и временной локальности.

Необходимо обратить внимание на следующий факт. При переключении контекста процессов нужно добиться того, чтобы новый процесс "не видел" в ассоциативной памяти информацию, относящуюся к предыдущему процессу, например очищать ее. Таким образом, использование ассоциативной памяти увеличивает время переключения контекста. Рассмотренная двухуровневая (ассоциативная память + таблица страниц) схема преобразования адреса является ярким примером иерархии памяти, основанной на использовании принципа локальности, о чем говорилось во введении к предыдущей лекции.

Лекция 8

Тема 1.5.2. Плоская модель сегментации

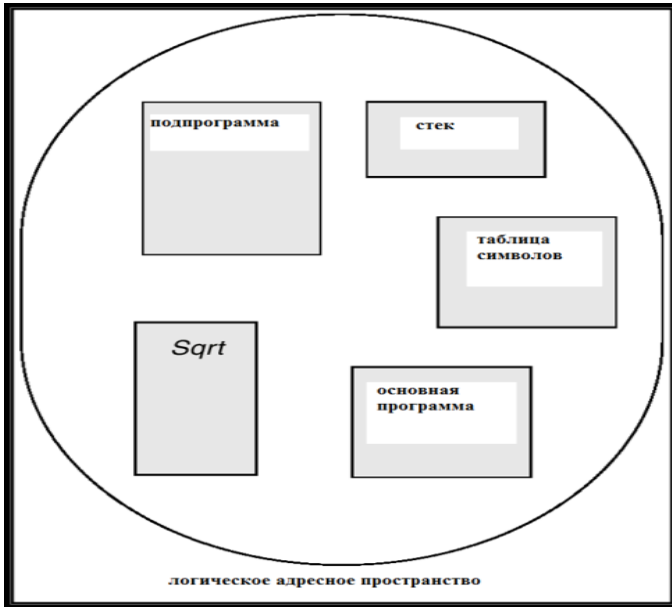
Введение

В лекции рассмотрена сегментная организация памяти – альтернатива страничной организации. Дано обоснование сегментной организации и ее связи с логической структурой программы. Рассмотрена смешанная – сегментно-страничная – организация памяти, применяемая во многих системах.

Принципы сегментной организации памяти

Сегментная организация памяти (segmentation) - схема распределения памяти в виде сегментов переменной длины, соответствующая пользовательской трактовке распределения памяти, т.е. логической структуре программ и данных. С точки зрения пользователя (разработчика программы), программа – это набор **модулей** кода и данных, каждому из которых должен соответствовать свой **сегмент** в памяти. Сегмент – логическая единица распределения памяти, предназначенная для размещения в памяти одного модуля программного кода или данных. Например, в виде сегментов памяти могут быть представлены:

- основная программа;
- процедура;
- функция;
- метод;
- объект;
- набор локальных переменных;
- набор глобальных переменных;
- общий блок данных (например, COMMON-блок в языке FORTRAN);
- стек;
- таблица символов;
- массив.



Архитектура сегментной организации памяти

Многие принципы архитектуры сегментной организации схожи с принципами страничной организации, однако во всех случаях приходится учитывать, что длина сегмента переменна, и хранить ее в явном виде в таблицах.

Регистр длины таблицы сегментов - segment-tablelengthregister (STLR) содержит число сегментов, используемое программой.

Номер сегмента s корректен, если $s < \text{STLR}$.

Перемещение (relocation) программ и данных при сегментной организации **динамическое**, т.е. выполняется во время исполнения программы с помощью таблицы сегментов.

Возможен общий доступ (sharing) нескольких процессов к одному и тому же сегменту, т.е. поддерживается концепция **разделяемых сегментов**. При

этом логический номер общего сегмента для разных процессов будет одним и тем же.

Стратегии распределения памяти при сегментной организации: **метод первого подходящего** или **метод наиболее подходящего**. Метод наименее подходящего при сегментной организации смысла не имеет, так как он не улучшает ситуацию с фрагментацией (ввиду переменной длины сегментов). Соответственно, при сегментной организации возможна **внешняя фрагментация**, для борьбы с которой применяется компактировка.

Защита при сегментной организации организована аналогично защите при страничной организации, однако, ввиду того, что каждый сегмент выполняет определенную логическую функцию в программе, дополнительно с каждым сегментом связываются его **признаки защиты**. Таким образом, в каждом элементе таблицы сегментов хранятся:

- **validation-бит** (аналогично страничной организации): значение бита, равное 0, означает, что сегмент неверный, т.е. не принадлежит логической памяти процесса;
- **полномочия чтения (read) / записи (write) / исполнения (execute)** – каждое кодируется одним битом; значение бита, равное 0, означает, что процесс не имеет данных полномочий.

Например, если сегмент является сегментом данных, то система устанавливает в таблице сегментов бит защиты от исполнения равным 0. Если это сегмент кода, то целесообразна установка в 0 битов защиты от чтения и от записи.

Биты защиты связываются с сегментами. Совместный доступ к коду осуществляется на уровне сегментов.

В системе "Эльбрус" к стандартному набору признаков защиты был добавлен еще один: **защита от записи в сегмент адресной информации** (данный признак целесообразен, если, например, сегмент предназначен для записи в файл).

Поскольку сегменты различаются по длине, распределение памяти в виде сегментов – это общая задача динамического распределения памяти.

На рис. приведена схема адресации при сегментном распределении памяти. Логический адрес (s, d), где s – номер сегмента, d – смещение внутри сегмента, обрабатывается следующим образом. По номеру сегмента s происходит обращение в таблицу сегментов, и определяется база сегмента – его начальный адрес в основной памяти. Смещение d сравнивается с длиной (границей) сегмента. Если оно меньше, то оно складывается с базой, и в результате получается физический адрес, по которому и происходит обращение в память. Если смещение больше или равно базе, происходит прерывание – ошибка адресации. На схеме не показана еще одна проверка – для номера сегмента s проверяется,

что он не превосходит значения регистра длины таблицы сегментов, иначе – прерывание.

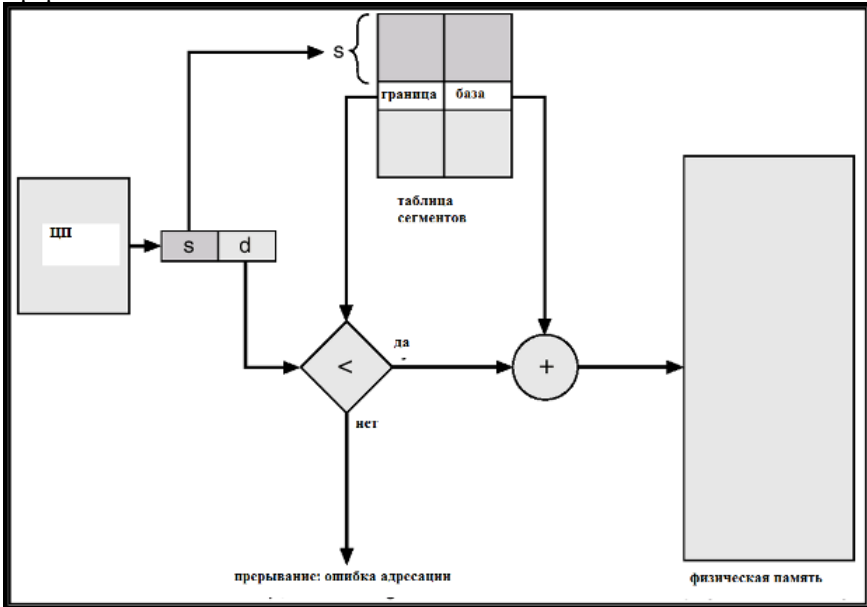


Схема адресации при сегментной организации памяти.

Пользователи систем UNIX, Linux, Solaris наверняка по ассоциации вспомнят часто встречающееся системное сообщение: **Segmentation violation (нарушение сегментации)** при прерывании. Причина подобного сообщения обычно следующая: программа пытается обратиться по **пустому (нулевому) указателю**, номер сегмента в котором, естественно, равен 0, а номер сегмента 0 считается в системе недопустимым. Таким образом, через проверку номера сегмента, UNIX защищает от такой часто встречающейся ошибки, как обращение по пустому указателю. Аналогично, по той же самой причине, если программа пытается практически любое другое число (например, 5) рассматривать как адрес и обращаться по нему, то также генерируется прерывание и сигнал **SIGSEGV (segmentation violation)**.

Сегментно-страничная организация памяти в системах MULTICS и "Эльбрус"

Сегментное распределение памяти, как видно из предыдущих пунктов, имеет свои неоспоримые преимущества. Однако, к сожалению, проблема внешней фрагментации при чисто сегментной организации памяти весьма серьезна.

Поэтому в системе MULTICS проблемы внешней фрагментации и длительного времени поиска решены путем страничной организации памяти

для сегментов. Данное решение отличается от чисто сегментной организации тем, что элемент таблицы сегментов содержит не базовый адрес сегмента, а базовый адрес **таблицы страниц** для данного сегмента.

В системе "Эльбрус" данные проблемы решены по-другому: в системе используется страничная организация для **математической (виртуальной, логической)** памяти и сегментная организация – для физической памяти. Математическая память (т.е. образ памяти на диске) распределяется с точностью **до страницы**, а физическая – с точностью **до слова**. Кроме того, как уже отмечалось, для адресации в системе "Эльбрус" используются **дескрипторы массивов**, содержащие начальный адрес, длину и признаки защиты массива.

Логический адрес (s, d) используется следующим образом. Номер сегмента s складывается с содержимым регистра STBR, после чего происходит обращение по полученному адресу в таблицу сегментов. Смещение d сравнивается с длиной сегмента; если оно больше или равно длине сегмента, то прерывание. Из элемента таблицы сегментов извлекается адрес базы таблицы страниц данного сегмента, который складывается со старшими разрядами смещения (p). Младшие разряды смещения используются как смещение внутри страницы, адрес которой извлекается из таблицы страниц данного сегмента.

Нельзя не отметить, что данная схема, на наш взгляд, несколько усложнена. Система MULTICS, в которой этот и многие другие механизмы доведены, казалось бы, до совершенства, вызвала противоположную тенденцию в развитии ОС – к упрощению. Как мы уже говорили, даже название следующей системы – UNIX – выбрано как противоположность MULTICS, что говорит о многом. Таковы тенденции развития во многих областях, в том числе и в операционных системах.

Сегментно-страничная организация памяти в системе Intel 386

В более современных системах, чем MULTICS, также используется сегментно-страничная организация. Например, в системе Intel 386 используется сегментно-страничная организация памяти с двухуровневой схемой страничной организации

Ключевые термины

Базовый регистр таблицы сегментов - segment-tablebase register (STBR) – регистр, содержащий адрес таблицы сегментов в памяти.

Логический адрес при сегментной организации памяти - пара:

<segment-number, offset>, где segment-number – номер сегмента, offset – смещение в сегменте.

Признаки защиты – информация для защиты сегмента в элементе таблицы сегментов: **validation-бит** (признак корректности номера сегмента для процесса), **биты защиты от записи, от чтения и от исполнения**.

Разделяемые (общие) сегменты – сегменты, общие для нескольких процессов и имеющие одинаковые логические номера в их таблицах сегментов.

Регистр длины таблицы сегментов - segment-tablelengthregister (STLR) – регистр, содержащий число сегментов, используемое программой.

Сегмент – логическая единица распределения памяти, предназначенная для размещения в памяти одного модуля программного кода или данных.

Сегментная организация памяти (segmentation) - схема распределения памяти в виде сегментов переменной длины, соответствующая пользовательской трактовке распределения памяти, т.е. логической структуре программ и данных.

Сегментно-страничная организация памяти – модификация сегментной организации памяти для борьбы с фрагментацией, основанная на страничной организации каждого сегмента.

Таблица сегментов – системная таблица для отображения логических адресов в физические при сегментной организации памяти. Каждый ее элемент содержит начальный адрес сегмента в физической памяти, длину сегмента и **признаки защиты**.

Краткие итоги

Сегментная организация памяти – стратегия распределения памяти сегментами переменной длины, каждый из которых выполняет определенную логическую функцию в программе как модуль программы или данных – стек, массив, подпрограмма и т.д.

При сегментной организации логический адрес имеет вид: (номер сегмента, смещение внутри сегмента). Организуется системная таблица сегментов, каждый элемент которой содержит базовый адрес сегмента, его длину и признаки защиты – validation-бит, определяющий корректность номера сегмента, биты защиты от записи, чтения и исполнения. Базовый регистр таблицы сегментов содержит начальный адрес таблицы сегментов, регистр длины таблицы сегментов содержит ее длину.

Перемещение при сегментной организации осуществляется динамически, во время выполнения программы.

Возможен общий доступ нескольких процессов к одному и тому же сегменту с одним логическим номером (разделяемые сегменты).

Стратегии распределения памяти при сегментной организации – методы первого подходящего и наиболее подходящего. Возможна внешняя фрагментация.

При адресации по логическому адресу для сегментной организации смещение внутри сегмента сравнивается с длиной сегмента из элемента таблицы сегментов. В случае превышения – прерывание.

Для борьбы с фрагментацией в некоторых системах (MULTICS, "Эльбрус", Intel 386) применяется смешанная, сегментно-страничная организация памяти, при которой для каждого сегмента организуется собственная таблица страниц.

Лекция 9

Тема 1.5.3. Кэширование

Кэширование. Методы загрузки и замещения страниц

- Кэш транслированных адресов
- Методы поиска в TLB
- Кэширование памяти и TLB
- Методы загрузки страниц
- Методы замещения страниц
- Реализации LRU стратегии

Кэш

Копия данных, к которой доступ осуществляется быстрее, чем к оригинальным данным. Ускоряет доступ к часто используемым данным и не влияет на время доступа к редко используемым данным.

Кэширование - фундаментальный принцип в современных системах. Применяется повсеместно: при трансляции адресов, доступе к памяти, файлам, сетевым адресам, в процессе авторизации доступа и т.п.

Кэш транслированных адресов

Механизм трансляции замедляет доступ к памяти. Трансляция выполняется очень часто: 1-2 раза при выполнении машинной команды (выборка команды, выборка операнда).

Часто обращения происходят к одной и той же странице памяти. При этом (для простой страничной схемы) один и тот же номер виртуальной страницы транслируется в номер физической страницы многократно. Не хотелось бы повторять трансляцию близких адресов по полной схеме через таблицу страниц.

Устройство управления памятью современных процессоров поддерживает небольшой аппаратный буфер для хранения недавно транслированных номеров страниц: TLB (TranslationLookasideBuffer). Время выборки из этого буфера 5-10 наносекунд. время выборки из памяти -сотни наносекунд.

Пример TLB для простой страничной памяти

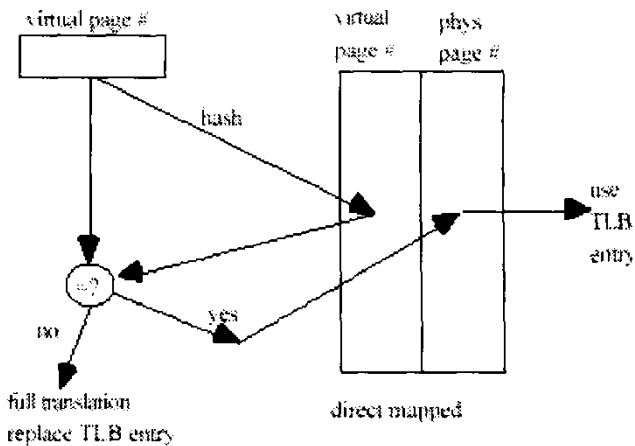
Номер виртуальной страницы	Номер физической страницы	Флажки
2	1	valid, rw
-	-	invalid
0	4	valid, rw

В первую очередь просматривается TLB и если нужного адреса нет, то осуществляется трансляция по полной схеме.

Методы поиска в tlb

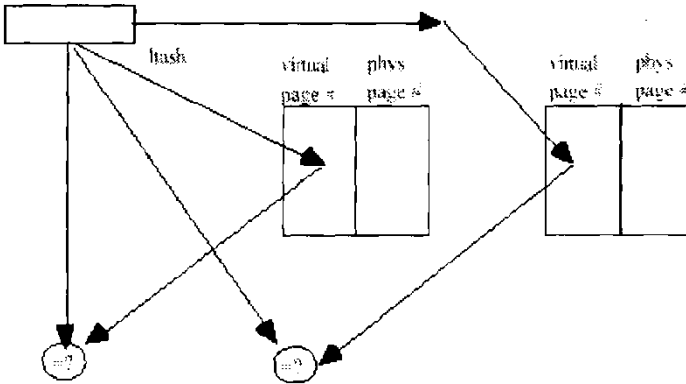
1. Последовательный поиск по TLB. Долго.

2. Прямое отображение. Номеру виртуальной страницы ставится в соответствие (хэш функция) слот TLB. Если два номера отображаются на один слот, то могут возникнуть проблемы. например, если выполняемая страница кода конфликтует со страницей стека. Хэш функция призвана минимизировать вероятность такого рода конфликтов.

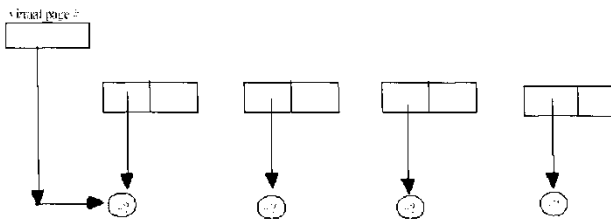


3. Ассоциативные таблицы. Поиск проводится параллельно по всем таблицам и если искомый элемент находится в одной из таблиц, то трансляция не выполняется. Чем больше уровень ассоциативности (больше параллельных таблиц), тем меньше вероятность конфликта, т. к. два требуемых одновременно элемента могут быть размещены в разных буферах.

virtualpage #



4. Полностью ассоциативные таблицы. Размер буфера = одному элементу.



Методы работы с TLB и с другими видами кэшей очень похожи. Разница состоит в уровне ассоциативности и методах замещения элементов в кэше. TLB обычно реализуются в виде полностью ассоциативных таблиц.

Методы замещения для tlb.

Если применяется прямое отображение, то выбора для замещения нет. Замещается тот элемент на который осуществлено отображение.

Для ассоциативных таблиц есть выбор: в каком буфере выполнять замещение. Возможные методы замещения: **дольше всех не используемый элемент, случайным образом, давно не используемый.** Для TLB часто

применяют метод случайного замещения. Этот метод прост и не требует хранения дополнительной информации об обращениях к элементу.

Примечание. При переключении контекста между процессами необходимо полностью сбрасывать TLB (Виртуальные номера страниц разных процессов могут совпадать). Однако, если при вызове хэш функции добавить к номеру страницы идентификатор процесса, то можно избежать сброса TLB при переключении контекста.

Тема 1.6.Файловая система и ввод и вывод информации.

Лекция 10

Тема 1.6.1Файловая система и ввод и вывод информации.

Одной из главных задач ОС является обеспечение обмена данными между приложениями и периферийными устройствами компьютера. Собственно ради выполнения этой задачи и были разработаны первые системные программы, послужившие прототипами операционных систем. В современной ОС функции обмена данными с внешними устройствами выполняет подсистема ввода - вывода. Клиентами этой подсистемы являются не только пользователи и приложения, но и некоторые компоненты самой ОС, которой требуется получение системных данных или их вывод, например подсистеме управления процессами при смене активного процесса необходимо записать на диск контекст приостанавливаемого процесса и считать с диска компонент активизируемого процесса.

Основными компонентами системы ввода — вывода являются драйверы, управляющие внешними устройствами, и файловая система. К подсистеме ввода - вывода можно также с некоторой долей условности отнести и диспетчер прерываний, рассмотренный выше. Условность заключается в том, что диспетчер прерываний обслуживает не только модули подсистемы ввода - вывода, но и другие модули ОС, в частности такой важный модуль, как планировщик - диспетчер потоков. Но из - за того, что планирование работ подсистемы ввода - вывода составляет основную долю нагрузки диспетчера прерываний, его вполне логично рассматривать как её составную часть (к тому же' первопричиной появления в компьютерах системы прерываний были в своё время именно операции с устройствами ввода — вывода).

Файловая система ввиду её сложности, специфичности и важности как основного хранилища всей информации вычислительной системы заслуживают отдельного рассказа.

Тем не менее, здесь файловая система рассматривается совместно с другими компонентами подсистемы ввода - вывода по двум причинам. В - первых, файловая система активно использует остальные части подсистемы ввода - вывода, а во - вторых, модель файла лежит в основе большинства механизмов доступа к устройствам, используемым в подсистемах ввода - вывода.

Задачи ос по управлению файлами и устройствами

Подсистема ввода - вывода (Input - OutputSubsystem) мультипрограммной ОС при обмене данными с внешними устройствами компьютера должна решать ряд общих задач, из которых наиболее важными являются следующие:

- Организация параллельной работы устройств ввода - вывода и процессора;
 - Согласование скоростей обмена и кэширование данных;
 - Разделение устройств и данных между процессами;
 - Обеспечение удобного логического интерфейса между устройствами и остальной частью системы;
 - Поддержка широкого спектра драйверов с возможностью простого включения в систему нового драйвера;
 - Динамическая загрузка и выгрузка драйверов;
 - Поддержка нескольких файловых систем;
 - Поддержка синхронных и асинхронных операций ввода-вывода;
- Организация параллельной работы устройств ввода - вывода и процессора*

Каждое устройство ввода – вывода вычислительной системы – диск, принтер, терминал и т. п. – снабжено специализированным блоком управления, называемым контроллером. Контроллер взаимодействует с драйвером – системным, программным модулем, предназначенным для управления данным устройством. Контроллер периодически принимает от драйвера выводимую на устройство информацию, а также команды управления, которые говорят о том, что с этой информацией нужно сделать (например, вывести в виде текста в определённую область терминала или записать в определённый сектор диска). Под управлением контроллера устройство может некоторое время выполнять свои операции автономно, не требуя внимания со стороны центрального процессора. Это время зависит от многих факторов - объема выводимой информации, степени интеллектуальности управляющего устройством контроллера, быстродействия устройства и т. п. Даже самый примитивный контроллер, выполняющий простые функции, обычно тратит много времени на самостоятельную реализацию подобной функции после получения очередной команды процессора. Это же справедливо и для сложных контроллеров, так как скорость работы любого устройства ввода - вывода, даже самого скоростного, обычно существенно ниже скорости работы процессора.

Процессы, происходящие в контроллерах, протекают в периоды между выдачами команд независимо от ОС. От подсистемы ввода — вывода требуется спланировать в реальном масштабе времени (в котором работают внешние устройства) запуск и приостановку большого количества разнообразных драйверов, обеспечив приемлемое время реакции каждого драйвера на независимые события контроллера. С другой стороны, необходимо минимизировать загрузку процессора задачами ввода - вывода, оставив как можно больше процессорного времени на выполнение пользовательских потоков.

Данная задача является классической задачей планирования систем реального времени и обычно решается на основе приоритетной многоуровневой схемы обслуживания по прерываниям. Для обеспечения приемлемого уровня реакции все драйверы (или части драйверов) распределяются по нескольким приоритетным уровням в соответствии с требованиями ко времени реакции и временем использования процессора. Для реализации приоритетной схемы обычно задействуется общий диспетчер прерываний ОС.

Согласование скоростей обмена и кэширование данных

При обмене данными всегда возникает задача согласования скорости. Например, если один пользовательский процесс вырабатывает некоторые данные и передает их другому пользовательскому процессу через оперативную память, то в общем случае скорости генерации данных и их чтения не совпадают. Согласование скорости обычно достигается за счет буферизации данных в оперативной памяти и синхронизации доступа процессов к буферу.

В подсистеме ввода - вывода для согласования скоростей обмена также широко используется буферизация данных в оперативной памяти. В тех специализированных операционных системах, в которых обеспечение высокой скорости ввода - вывода является первоочередной задачей (управление в реальном времени, услуги сетевой файловой службы и т. п.), большая часть оперативной памяти отводится не под коды прикладных программ, а под буферизацию данных. Однако буферизация только на основе оперативной памяти в подсистеме ввода - вывода оказывается недостаточной - разница между скоростью обмена с оперативной памятью, куда процессы помещают данные для обработки, и скоростью работы внешнего устройства часто становится слишком значительной, чтобы в качестве временного буфера использовать оперативную память - ее объема может просто и не хватить. Для таких случаев необходимо предусмотреть особые меры, и часто в качестве буфера используется дисковый файл, называемый также спул - файлом (от spool - шпулька, тоже буфер, только для ниток). Типичный пример применения спулинга дает организация вывода данных на принтер. Для печатаемых документов объем в несколько десятков мегабайт не редкость поэтому для их временного хранения (а печать каждого документа занимает от нескольких минут до десятков минут) объема оперативной памяти явно недостаточно.

Другим решением этой проблемы является использование большой буферной памяти в контроллерах внешних устройств. Такой подход особенно полезен в тех случаях, когда помещение данных на диск слишком замедляет обмен (или когда данные вводятся на сам диск). Например, в контроллерах графических дисплеев применяется буферная память, соизмеримая по объему с оперативной, и это существенно ускоряет вывод графики на экран.

Буферизация данных позволяет не только согласовать скорости работы процессора и внешнего устройства, но и решить другую задачу — сократить количество реальных операций ввода - вывода за счет кэширования данных. Дисковый кэш является неизменным атрибутом подсистем ввода - вывода практически всех операционных систем, значительно сокращая время доступа к хранимым данным.

Разделение устройств и данных между процессами

Устройства ввода – вывода норм разделяется процессам, как в монопольное, так и в совместное (разделяемое) использование. При этом ОС должна обеспечивать контроль доступа теми же способами, что и при доступе процессов к другим ресурсам вычислительной системы - путем проверки прав пользователя или группы пользователей, от имени которых действует процесс, на выполнение той или иной операции над устройством. Например, определенной группе пользователей разрешено захватывать в монопольное владение последовательный порт, другим же пользователям это запрещено.

Операционная система может контролировать доступ не только к устройству в целом, но и к отдельным порциям данных, хранимых или отображаемых этим устройством. Диск является типичным примером устройства, для которого важно контролировать доступ не к устройству в целом, а к отдельным каталогам и файлам. При выводе информации на графический дисплей отдельные окна экрана также представляют собой ресурсы, к которым необходимо обеспечить тот или иной вид доступа для протекающих в системе процессов. При этом для каждой порции данных или части устройства могут быть заданы свои права доступа, не связанные прямо с правами доступа к устройству в целом. Так, в файловой системе обычно для каждого каталога и файла можно задать индивидуальные права доступа. Очевидно, что для организации совместного доступа к частям устройства или частям данных, хранящихся на нем, непременным условием является задание режима совместного использования устройства в целом.

Одно и то же устройство в разные периоды времени может использоваться как в разделяемом, так и в монопольном режимах. Тем не менее, существуют устройства, для которых обычно характерен один из этих режимов, например последовательные порты и алфавитно-цифровые терминалы чаще используются в монопольном режиме, а диски - в режиме совместного доступа. Операционная система должна предоставлять эти устройства в обоих режимах, осуществляя отслеживание процедур захвата и когда подсистема ввода - вывода не разрешает драйверу непосредственно взаимодействовать с аппаратурой контроллера, а выполняет эти операции самостоятельно. Экранирование драйвера от аппаратуры является весьма полезной функцией, так как драйвер в этом случае становится независимым от аппаратной платформы. Подсистема ввода - вывода может поддерживать несколько различных типов интерфейсов DKI/ DDI, представляя

специфический интерфейс для устройств определенного класса. Так в ОС Windows NT для драйверов сетевых адаптеров существует интерфейс стандарта NDIS (NetworkDriverInterfaceSpecification), в то время как драйверы сетевых транспортных протоколов взаимодействуют с верхними слоями сетевого программного обеспечения по интерфейсу TEM (Transport Driver Interface).

Обычно подсистема ввода - вывода поддерживает большое количество системных функций, которые драйвер может вызывать для выполнения некоторых типовых действий. Примерами могут служить упомянутые операции обмена с регистрами контроллера, ведение буферов для промежуточного хранения данных ввода - вывода, синхронизация работы нескольких драйверов, копирование данных из пользовательского пространства в пространство системы и т. д.

Для поддержки процесса разработки драйверов операционной системы обычно выпускается так называемый пакет DDK (Driver Development Kit), представляющий собой набор соответствующих инструментальных средств — библиотек, компиляторов и отладчиков.

Цели и задачи файловой системы

Файл - это именованная область внешней памяти, в которую можно записывать и из которой можно считывать данные. Файлы хранятся в памяти, на зависящей от энергопитания, обычно — на магнитных дисках. Однако нет правил без исключений. Одним из таких исключений является т. н. электронный диск, когда в оперативной памяти создаётся структура, имитирующая файловую систему.

Основные цели использования файла:

- Долговременное и надёжное хранение информации. Долговременность достигается за счет использования запоминающих устройств, не зависящих от питания, а высокая надёжность определяется средствами защиты доступа к файлам и общей организацией программного кода ОС, при которой сбои аппаратуры чаще всего не разрушают информацию, хранящуюся в файлах.

- Совместное использование информации. Файлы обеспечивают естественный и лёгкий способ разделения информации между приложениями и пользователями за счет наличия понятного человеку символического имени и постоянства хранимой информации и расположения файла. Пользователь всегда должен иметь удобные средства работы с файлами, включая каталоги справочники, объединяющие файлы в группы, средства поиска файлов по признакам, набор команд для создания, модификации и удаления файлов. Файл может быть создан одним пользователем, а затем использоваться совсем другим пользователем, при этом создатель файла или администратор могут определить права доступа к нему других пользователей. Эти цели реализуются в ОС файловой системой.

Файловая система (ФС) - это часть операционной; системы включающая:

- Совокупность всех файлов на диске;
- Наборы структур данных, используемых для управления файлами, такие, например, как каталоги файлов, дескрипторы файлов, таблицы распределения свободного и занятого пространства на диске.
- Комплекс системных программных средств, реализующих различные операции над файлами, такие как создание, уничтожение, чтение, запись, именование и поиск файлов.

Файловая система позволяет программам обходиться набором достаточно простых операций для выполнения достаточно простых операций над некоторым абстрактным объектом, представляющим файл. При этом программистам не нужно иметь дело с деталями действительного расположения данных на диске, буферизацией данных и другими низкоуровневыми проблемами передачи данных с низкоуровневого запоминающего устройства. Все эти функции файловая система берет на себя. Файловая система распределяет дисковую память, поддерживает именование файлов, отображает имена файлов и соответствующие адреса во внешней памяти, обеспечивает доступ к данным, поддерживает разделение, защиту и восстановление файлов.

Таким образом, файловая система играет роль промежуточного слоя, экранирующего все сложности физической организации долговременного хранилища данных, и создающего для программ более простую логическую модель этого хранилища, а также предоставляя им набор наиболее удобных в использовании команд для манипулирования файлами.

Задачи, решаемые ФС, зависят от способа организации вычислительного процесса в целом. Самый простой тип – ФС в однопользовательских и однопрограммных ОС, к числу которых относится, например MS - DOS.

Основные функции такой ОС нацелены на:

- Именование файлов.
- Программный интерфейс для приложений.
- Отображения логической модели файловой системы на физическую организацию хранилища данных.
- Устойчивость файловой системы к сбоям питания, ошибкам аппаратных и программных средств.

Задачи ФС усложняются в операционных однопользовательских мультипрограммных ОС, которые, хотя и предназначены для работы одного пользователя, но дают ему возможность запускать одновременно несколько процессов. Озониэ первых ОС этого типа стала OS/2. К перечисленным выше задачам добавляется новая задача совместного доступа к файлу из нескольких процессов. Файл в этом случае является разделяемым ресурсом, а

значит, файловая система должна решать весь комплекс проблем, связанных с такими ресурсами. В частности, в ФС должны быть предусмотрены средства блокировки файла и его частей, предотвращения гонок, исключение тупиков, согласование копий и т. п. В многопользовательских системах появляется ещё одна задача: защита файла одного пользователя от несанкционированного доступа другого пользователя. Ещё более сложными становятся функции ФС, которая работает в составе сетевой ОС.

Тема 1.7. Работа в операционных системах и средах

Лекция 11

Управление безопасностью.

Безопасность – одна из наиболее актуальных проблем в области ИТ в настоящее время, ввиду сильной зависимости повседневной деятельности и бизнеса от компьютерных технологий и ввиду резко возрастающего числа сетевых атак (киберпреступности). Особенно важна безопасность для операционных систем и сетей как основных объектов атак. В лекции рассмотрены следующие вопросы:

- Проблема безопасности
- Аутентификация
- Программные угрозы (атаки)
- Системные угрозы (атаки)
- Защита систем
- Обнаружение взлома
- Криптография
- Безопасность в Windows NT / 2000 / XP / 2003 / Vista, в .NET
- Инициатива Trustworthy Computing Initiative корпорации

Microsoft.

Проблема безопасности

Безопасность (security) – это защита от внешних атак. В настоящее время наблюдается значительный рост числа самых разнообразных атак хакеров, угрожающих целостности информации, работоспособности компьютерных систем и зависящих от них компаний, благосостоянию и личной безопасности людей. Для защиты от атак необходимы специальные меры безопасности, компьютерные технологии и инструменты.

В любой компьютерной системе должна быть реализована **подсистема безопасности**, которая должна проверять внешнее окружение системы и защищать ее от:

- Несанкционированного доступа
- Злонамеренной модификации или разрушения
- Случайного ввода неверной информации.

Практика показывает, что легче защитить от случайной, чем от злонамеренной порчи информации.

Аутентификация

Одной из наиболее широко используемых мер безопасности является **аутентификация (authentication)** – идентификация пользователей при входе в систему. Такая идентификация пользователей наиболее часто реализуется через **логины** – зарегистрированные имена пользователей для входа в систему – и **пароли** – секретные кодовые слова, ассоциируемые с каждым логином.

Основной принцип использования паролей в том, что они должны сохраняться в секрете. Поэтому одна из традиционных целей атакующих хакеров состоит в том, чтобы любыми способами выведать у пользователя его логин и пароль. Для сохранения секретности паролей предпринимаются следующие меры.

- **Частая смена паролей.** Аналогичные меры применялись в армии во время войны. Большинство сайтов и других систем (например, сайт партнеров фирмы Microsoft) требуют от пользователей регулярной (например, не реже, чем раз в три месяца) смены паролей, иначе сайт блокируется для доступа. Подобные меры вполне оправданы.

- **Использование "не угадываемых" паролей.** Практически все системы требуют от пользователя при регистрации устанавливать пароли, не являющиеся легко угадываемыми: например, как правило, пароль должен содержать большие и маленькие буквы и цифры, специальные символы и иметь длину не менее 7-8 символов. Используются также автоматические генераторы не угадываемых паролей. Поэтому использование в качестве паролей легко угадываемых слов – например, имени любимой собаки или общеупотребительного понятия – не рекомендуется.

- **Сохранение всех неверных попыток доступа.** Во многих системах реализован системный журнал, в котором фиксируются все неверные попытки ввода логинов и паролей. Обычно дается фиксированное число таких попыток (например, три).

Пароли также могут быть **закриптованы** или разрешены для доступа лишь один раз, после чего от пользователя требуется смена пароля.

Программные угрозы (атаки)

Рассмотрим некоторые типичные виды угроз и атак, используемые хакерами.

Троянская программа (TrojanHorse) – атакующая программа, которая "подделывается" под некоторую полезную программу, но при своем запуске не по назначению (злонамеренно) использует свое окружение, например, получает и использует конфиденциальную информацию. Троянские программы используют системные механизмы для того, чтобы программы, написанные одними пользователями, могли исполняться другими пользователями.

Вход в ловушку (TrapDoor) - использование логина или пароля, который позволяет избежать проверок, связанных с безопасностью.

Переполнение стека и буфера (StackandBufferOverflow) - использование ошибки в программе (переполнение стека или буферов в памяти) для обращения к памяти другого пользователя или процесса с целью нарушения ее целостности.

Системные угрозы (атаки)

Рассмотрим также некоторые типичные атаки, использующие **уязвимости (vulnerabilities)** в системных программах – ошибки и недочеты, дающие возможность организации атак.

Черви (Worms) – злонамеренные программы, использующие механизмы самовоспроизведения (размножения). Например, один из Интернет-червей использует сетевые возможности UNIX (удаленный доступ) и ошибки в программах **finger** и **sendmail**. Принцип его действия следующий: некоторая постоянно используемая в сети системная программа распространяет главную программу червя.

вирусы – фрагменты кода, встраивающиеся в обычные программы с целью нарушения работоспособности этих программ и всей компьютерной системы. В основном вирусы действуют на микрокомпьютерные системы. Вирусы скачиваются с публично доступных сайтов или с дисков, содержащих "инфекцию". Для предотвращения заражения компьютерными вирусами необходимо соблюдать принципы безопасности при использовании компьютеров (**safecomputing**) – использовать **антивирусы, guards** – программы, постоянно находящиеся в памяти и проверяющие на вирусы каждый открываемый файл - .exe, doc, и т.д.

Отказ в обслуживании (DenialofService – DoS) – одна из распространенных разновидностей атак на сервер, заключающаяся в создании искусственной перегрузки сервера с целью препятствовать его нормальной работе. Например, для Web-сервера такая атака может заключаться в том, чтобы искусственно сгенерировать миллион запросов "GET". Если сервер реализован не вполне надежно, подобная атака чаще всего приводит к переполнению памяти на сервере и необходимости его перезапуска.

Типы сетевых атак

Рассмотрим некоторые типы современных сетевых атак, которых необходимо постоянно остерегаться пользователям.

Phishing – попытка украсть конфиденциальную информацию пользователя путем ее обманного получения от самого пользователя. Даже само слово **phishing** – искаженное слово **fishing** (рыбная ловля), т.е. хакер с помощью этого приема как бы пытается поймать чересчур наивного пользователя "на удочку". Например, напугав в своем сообщении пользователя, что его логин и пароль, кредитная карта или банковский счет под угрозой, хакер пытается добиться от пользователя в ответ ввода и отправки некоторой конфиденциальной информации. Обычно phishing-сообщение по электронной почте приходит как бы от имени банка и подделывается под цвета, логотипы и т.д., используемые на сайте банка. Однако для его разоблачения обычно достаточно подвести курсор мыши (не кликая ее) к приведенной web-ссылке или email-адресу (при этом она высвечивается) и убедиться в том, что адрес указывает отнюдь не на банк, а на совершенно посторонний сайт или email. Поэтому пользователям не

следует быть слишком наивными. Другая действенная мера, если phishing происходит регулярно с одних и тех же email-адресов, - включить эти адреса в черный список на email-сервере. Тогда подобные сообщения вообще не будут доходить до входного почтового ящика пользователя.

Pharming – перенаправление пользователя на злонамеренный Web-сайт (обычно с целью phishing). Меры предотвращения со стороны пользователя мы уже рассмотрели. В современные web-браузеры встроены программы антифишингового контроля, которые запускаются автоматически при обращении к сайту. Хотя это отнимает у пользователя некоторое время, подобные меры помогают предотвратить многие атаки.

Tamperingwithdata – злонамеренное искажение или порча данных. Действенной мерой по борьбе с подобными атаками является **криптование** информации.

Spoofing – "подделка" под определенного пользователя (злонамеренное применение его логина, пароля и полномочий). Логин и пароль при этом либо получены от пользователя обманным путем (например, в результате phishing), либо извлечены из "взломанного" хакерской программой системного файла.

Elevationofprivilege – попытка расширить полномочия (например, до полномочий системного администратора) с целью злонамеренных действий. Поэтому наиболее секретная информация в любой компьютерной системе – пароль системного администратора, который необходимо защищать особенно тщательно.

TrustworthyComputing (TWC) Initiative

Инициатива под таким названием (инициатива надежных и безопасных вычислений) объявлена в 2002 г. в историческом электронном письме основоположника корпорации Microsoft Билла Гейтса всем сотрудникам компании. Основная суть инициативы TWC заключается в том, что безопасности необходимо уделять особое внимание при разработке программной системы, начиная с самых ранних этапов. Однако этим инициатива TWC не исчерпывается – смысл и цели ее гораздо шире и охватывают также экономические, юридические аспекты и "человеческий фактор".

Основные принципы инициативы TWC:

Безопасность (Security) – реализация и использование в любой программной системе действенных мер защиты от внешних атак; использование специальных методов разработки программ, направленных на достижение этой цели.

Сохранение конфиденциальности информации (Privacy) – использование программным обеспечением частной и корпоративной информации только с явного согласия пользователя и только для понятных

ему законных целей; защита конфиденциальной информации от взлома в результате атаки.

Надежность (Reliability) – предсказуемость поведения программных систем, которые должны обеспечивать в заданном окружении ожидаемое правильное поведение программы.

Оперативность, законность и корректность бизнеса (BusinessIntegrity) – оперативность работы группы сопровождения программного продукта и своевременные консультации пользователей по вопросам безопасности; корректность бизнеса компании – разработчика программного обеспечения.

Сама корпорация Microsoft с 2002 г. полностью реорганизовала бизнес-процессы разработки программного обеспечения, используя новую схему жизненного цикла для разработки безопасных программ – **SDLC (SecurityDevelopmentLifeCycle)**. Принципы TWC воплощены во всех новых версиях продуктов Microsoft: Internet Explorer 7 и 8, Windows Vista и др.

Microsoft своей инициативой TWC призвала все остальные компании и индивидуальных разработчиков следовать предложенным ею принципам, хотя изначально отношение к инициативе TWC в мире было достаточно осторожным и даже скептическим.

Microsoft финансировала работы по обеспечению TWC и обучение TWC в университетах.

Следует отметить, что обучение TWC в университетах в мире только начато. Наибольшее внимание этим вопросам в первую очередь уделяют университеты военного подчинения и назначения.

Одним из первых университетов, обучающих студентов ИТ-специальностей принципам TWC, является математико-механический факультет СПбГУ – более подробное изучение TWC в курсе проф. В.О. Сафонова "Архитектуры и модели программ и знаний" (4 курс) и изложение элементов TWC во всех других курсах проф. В.О. Сафонова: "Операционные системы и сети", "Java-технология", "Компиляторы".

Лекция 12

Тема 1.7.2. Требования к аппаратным ресурсам

Аппаратными средствами (Hardware) называют совокупность всех устройств, которые составляют компьютер или могут к нему добавляться по мере необходимости.

К важнейшим аппаратным средствам относятся системный блок и периферийные устройства.

По способу расположения устройств относительно центрального процессорного устройства (ЦПУ –

Central Processing Unit, CPU) различают внутренние и внешние устройства. Внешними, как правило, являются большинство устройств ввода-вывода данных (их также называют периферийными устройствами) и некоторые устройства, предназначенные для длительного хранения данных.

Схема фон Неймана

Выдающийся математик Джон фон Нейман в середине 20 века, анализируя работу первых ЭВМ, пришел к выводу о необходимости хранения выполняющейся программы и обрабатываемых по этой программе данных внутри машины, в ее электронных схемах, а не вне ее – на перфокартах, перфолентах или разьемах со штекерами.

Джон фон Нейман предложил новые принципы создания компьютеров, состоящие в следующем:

Принцип двоичного кодирования: вся информация, поступающая в ЭВМ, кодируется с помощью двоичных сигналов. Принцип программного управления: программа состоит из набора команд, которые выполняются процессором автоматически друг за другом в определенной последовательности. Принцип однородности памяти: программы и данные хранятся в одной и той же памяти, поэтому ЭВМ не различает, что хранится в данной ячейке памяти – число, текст или команда. Над командами можно выполнять такие же действия, как и над данными. Принцип адресности: структурно основная память состоит из пронумерованных ячеек; процессору в произвольный момент времени доступна любая ячейка.

Согласно фон Нейману, ЭВМ состоит из следующих основных блоков:

- устройство ввода/вывода информации;
- память компьютера;
- процессор, состоящий из устройства управления (УУ), через которое идет поток команд и данных, и арифметико-логического устройства (АЛУ), производящего арифметические и логические операции.

В общих чертах идеи фон Неймана и созданная им архитектура ЭВМ актуальны до сегодняшнего времени

Архитектура ЭВМ.

Под архитектурой ЭВМ понимается совокупность сведений об основных устройствах компьютера и их назначении, о способах представления программ и данных в машине, об особенностях ее организации и функционирования.

Электронно-вычислительная машина (ЭВМ), или компьютер, – это электронное устройство, используемое для автоматизации процессов приема, хранения, обработки и передачи информации, которые осуществляются по заранее разработанным человеком программам.

В состав любого современного компьютера входят:

- память – группа устройств, которые обеспечивают хранение данных и программ;
- процессор – одно или несколько устройств, которые обеспечивают задаваемую программой обработку данных;
- устройства ввода-вывода – группа устройств, которые обеспечивают обмен, то есть прием и передачу данных между пользователем и машиной или между двумя или более машинами.

Различные устройства компьютера подсоединяются друг к другу с помощью стандартизированных и унифицированных (то есть единообразных) аппаратных средств – кабелей, разъемов и т. д. При этом устройства обмениваются друг с другом информацией и управляющими сигналами, которые также приводятся к некоторым стандартным формам. Совокупность этих стандартных средств и форм образует конкретный интерфейс (interface – взаимный вид) того или иного устройства или компьютера в целом.

Интерфейс – совокупность унифицированных стандартных соглашений, аппаратных и программных средств, методов и правил взаимодействия устройств или программ.

Основные устройства ЭВМ.

Персональный компьютер – универсальная техническая система. Его конфигурацию (состав оборудования) можно гибко изменять по мере необходимости. Тем не менее, существует понятие базовой конфигурации, которую считают типовой. В таком комплекте компьютер обычно поставляется. Понятие базовой конфигурации может меняться. В настоящее время в базовой конфигурации рассматривают четыре устройства:

- системный блок;
- монитор;
- клавиатура;
- мышь.

Системный блок

Системный блок представляет собой основной узел, внутри которого установлены наиболее важные компоненты. Устройства, находящиеся внутри системного блока называют внутренними, а устройства, подключаемые к

нему снаружи, называют внешними. Внешние дополнительные устройства, предназначенные для ввода, вывода и длительного хранения данных, также называют периферийными.

Системный блок содержит все основные элементы ЭВМ – материнская плата, центральный процессор, оперативная память, системная шина, различные дисковые устройства, блок питания и многие другие. На лицевой панели системного блока размещаются: кнопка включения электропитания (Power - мощность), кнопка перезапуска (Reset - переустановка), ряд сигнальных лампочек и панели управления различных дисковых устройств. На задней панели находятся стандартные разъемы для крепления всевозможных дополнительных устройств и устройств ввода-вывода.

Основные компоненты системного блока

Блок питания. Материнскую плату. ВЗУ.

Блок питания

Преобразовывает переменный ток сетевого напряжения в постоянный ток низкого напряжения. В корпусе блока питания имеется встроенный вентилятор для охлаждения внутренних электрических схем. Часто в блоке питания входит розетка для питания монитора.

Материнская плата (motherboard)

Плата – это обычно прямоугольная пластина, используемая для выполнения монтажа необходимых электрических цепей, а также имеющая специальные разъемы для крепления шины и микросхем памяти, процессора и т. д.

Основная, самая большая по габаритам плата, располагающаяся внутри корпуса, называется материнской. На этой плате монтируется большинство остальных компонент ЭВМ.

1. Центральный процессор (CPU – Central Processor Unit) с охлаждающим вентилятором (cooler).

Процессор или микропроцессор – основное устройство компьютера, которое обеспечивает задаваемую программой обработку данных.

В общем случае процессор состоит из следующих компонентов:

- Арифметико-логическое устройство;
- Математический сопроцессор для обработки чисел с «плавающей»

точкой;

- Шины данных и адреса;
- Счетчики команд;
- Сверхбыстрая кэш-память малого объема.

Основные функции: действия по обработке данных и управление последовательностью выполнения таких действий.

Для ускорения выполнения машинных команд в процессоре предусмотрена регистровая память. Регистр – это устройство для кратковременного хранения информации в процессе ее обработки.

Характеристики процессора:

- Тактовая частота – количество тактовых импульсов, вырабатываемых тактовым генератором в секунду. Тактовые импульсы поступают на все остальные устройства компьютера и т. о. синхронизируют работу.

- Длина машинного слова – количество байтов в машинном слове. Машинное слово – наибольшая группа байтов, которая может быть обработана процессором за один машинный такт.

2. Оперативная память (RAM – RandomAccessMemory – память с произвольным доступом или прямоадресуемая память) или оперативное запоминающее устройство (ОЗУ).

Оперативная память – устройство компьютера, предназначенное для хранения выполняющихся в текущий момент времени программ, а также данных, необходимых для их выполнения.

Отличительные особенности – энергозависимость (при отключении электропитания вся информация, которая хранилась в оперативной памяти, безвозвратно теряется) и относительно высокая стоимость.

Для современных персональных компьютеров считается нормальным объемом памяти порядка 128-256 Мб. Если же работа связана с обработкой графики или звука, то лучше иметь объем памяти Мб.

3. Набор микросхем (chipset) отвечает за работу материнской платы. Обычно чипсет содержит в себе контроллеры прямого доступа к памяти, прерываний, таймеры, систему управления памятью и шиной. Чипсет – главная деталь материнской платы. От него зависят такие характеристики платы, как типы поддерживаемых процессоров, внешних устройств, модулей памяти (и возможности их сочетания) и системной шины. Иногда чипсет включает в себя интегрированные аудио - и видеокарты.

4. Системная шина, просто шина или магистраль – комплекс, состоящий из пучка проводов и электронных схем, обеспечивающих правильную передачу информации внутри компьютера. Главными характеристиками шины являются ее разрядность – число проводов в шине и частота работы.

5. Часто на материнской плате монтируется кэш-память (cache – запас, тайный склад) – высокоскоростная память, в которую копируются данные из оперативной памяти с целью оптимизации работы процессора с большими объемами данных. Обеспечивает повышение скорости вычислений.

6. Постоянная память, ПЗУ (постоянное запоминающее устройство), или ROM (ReadOnlyMemory – память только для чтения). Запись информации

в ПЗУ осуществляется только один раз на заводе-изготовителе. И в дальнейшем из этой памяти возможно только чтение.

При отключении электропитания данные, записанные в ПЗУ, сохраняются. Постоянная память используется для хранения наиболее важных и часто используемых служебных программ, которые осуществляют проверку работы отдельных устройств компьютера (тестирование), а также выполняют постоянно используемые операции по обмену данными между клавиатурой, монитором и памятью компьютера. Этот комплекс программ образует BIOS (BasicInput-OutputSystem) – базовая система ввода-вывода, содержащая основное программное обеспечение платы и программу самотестирования платы. BIOS получает управление при включении системной платы, выполняет самотестирование ее основных устройств, затем загружает операционную систему.

7. Энергонезависимая память на основе CMOS (ComplementaryMetal-Oxide-Semiconductor – структура типакомплементарный металл-оксид-полупроводник), а также аккумулятор для ее питания. Эта память содержит основные параметры настройки материнской платы, в частности тип и число накопителей на жестких дисках (винчестерах), наличие-отсутствие дисководов, пароль на загрузку компьютера.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Батаев, А. В. Операционные системы и среды / А.В. Батаев, Н.Ю. Налютин. — Москва: «Академия», 2019. — 272 с. — ISBN 978-5-4468-8118-5. — Текст: непосредственный.
2. Авдеев В.А. Периферийные устройства. Интерфейсы, схемотехника, программирование / В.А. Авдеев. — Электрон. текстовые данные. — Саратов: Профобразование, 2017. — 848 с. — ISBN 978-5-4488-0053-5. — Электрон. текстовые данные: URL: <http://www.iprbookshop.ru/63578.html> - Режим доступа: для зарегистрированных пользователей. — Текст: электронный.
3. Борисов Р.С. Информатика (базовый курс) [Электронный ресурс]: учебное пособие / Р.С. Борисов, А.В. Лобан. — Электрон. текстовые данные. — М.: Российский государственный университет правосудия, 2014. — 304 с. — ISBN 978-5-93916-445-0. — URL: <http://www.iprbookshop.ru/34551.html> - Режим доступа: для зарегистрированных пользователей. — Текст: электронный.
4. Галас В.П. Вычислительные системы, сети и телекоммуникации. Часть 1. Вычислительные системы электронный учебник / В.П. Галас. — Электрон. текстовые данные. — Владимир: Владимирский государственный университет им. А.Г. и Н.Г. Столетовых, 2016. — 232 с. — ISBN 2227-8397. — URL: <http://www.iprbookshop.ru/57363.html> - Режим доступа: для зарегистрированных пользователей. — Текст: электронный.
5. Галас В.П. Вычислительные системы, сети и телекоммуникации. Часть 2. Сети и телекоммуникации электронный учебник / В.П. Галас. — Электрон. текстовые данные. — Владимир: Владимирский государственный университет им. А.Г. и Н.Г. Столетовых, 2016. — 311 с. — ISBN 2227-8397. — URL: <http://www.iprbookshop.ru/57364.html> - Режим доступа: для зарегистрированных пользователей. — Текст: электронный.
6. Катунин Г.П. Основы инфокоммуникационных технологий Учебник / Г.П. Катунин. — Электрон. текстовые данные. — М.: Ай Пи Эр Медиа, 2018. — 304 с. — ISBN 978-5-93916-445-0. — URL: <http://www.iprbookshop.ru/74561.html> - Режим доступа: для зарегистрированных пользователей. — Текст: электронный
- Киселев С.В. Оператор ЭВМ: учеб. пособие для студ. учреждений сред. проф. образования /. — 7-е изд., испр. — М.: Издательский центр «Академия», 2014.
7. Ковган Н.М. Компьютерные сети: учебное пособие / Н.М. Ковган. — Электрон. текстовые данные. — Минск: Республиканский институт профессионального образования (РИПО), 2014. — 180 с. — ISBN 978-

- 985-503-374-6. — URL: <http://www.iprbookshop.ru/67638.html> - Режим доступа: для зарегистрированных пользователей. — Текст: электронный.
8. Коньков, К. А. Устройство и функционирование ОС Windows. Практикум к курсу Операционные системы. / Учебное пособие // К.А. Коньков. М.: Бином, Лаборатория знаний Интуит, 2013. - ISBN 978-985-503-421-6 – Текст: непосредственный.
 9. Куль Т.П. Операционные системы: учебное пособие / Т.П. Куль. — Электрон. текстовые данные. — Минск: Республиканский институт профессионального образования (РИПО), 2015. — 312 с. — ISBN 978-985-503-460-6. — <http://www.iprbookshop.ru/67677.html> - Режим доступа: для зарегистрированных пользователей. — Текст: электронный.
 10. Колмыкова Е.А., Кумскова. - Информатика: учеб. пособие для студ. проф. образования. – 7-е изд., стер. / Е.А. Колмыкова, Кумскова – М.: Издательский центр «Академия», 2010. – 416 с. ISBN 984-5-4488-0043-5 – Текст: непосредственный.
 11. Левчук Е.А. Технологии организации, хранения и обработки данных: учебное пособие / Е.А. Левчук. — Электрон. текстовые данные. — Минск: Вышэйшая школа, 2007. — 240 с. — ISBN 978-985-06-1409-4. — URL: <http://www.iprbookshop.ru/24081.html> - Режим доступа: для зарегистрированных пользователей. — Текст: электронный.
 12. Лобан А.В. Информатика (создание сайтов в сети Интернет): практикум для ФНО / А.В. Лобан. — Электрон. текстовые данные. — М.: Российский государственный университет правосудия, 2014. — 96 с. — ISBN 978-5-93916-405-4. — URL: <http://www.iprbookshop.ru/34552.html>
 13. Михеева Е.В. Информатика: учебник для студ. учреждений сред. проф. образования. – 5-е изд., стер. / Е.В. Михеева – М.: Издательский центр «Академия», 2010. – 352 с. ISBN 978-5-5466-0053-5– Текст: непосредственный.